



Разработчик event-driven микросервисов gRPC, Kafka, Redis

Модуль 1 - Введение в Event-driven архитектуру

Урок 1 - Что такое событийно-ориентированная архитектура

Шаг 01:

Вопрос 1

Ваш сервис обрабатывает заказы. Когда пользователь оформил заказ, нужно, чтобы склад и система уведомлений тоже узнали об этом. Что лучше всего описывает событийно-ориентированную архитектуру в этой ситуации?

Подсказка: Подумайте, что здесь является триггером — событие или запрос?

Ответ:

- **Сервис заказов публикует событие, а подписчики реагируют**
- Сервис заказов напрямую вызывает склад и уведомления
- Все сервисы пишут напрямую в одну таблицу базы данных
- Склад регулярно опрашивает сервис заказов

Шаг 02:

Вопрос 2

Вы внедряете Event-driven архитектуру в e-commerce. Какую основную роль выполняет Kafka?

Подсказка: Вспомните, где хранится поток сообщений и кто его обрабатывает.

Ответ:

- **Брокер сообщений, обеспечивающий доставку событий**
- Фреймворк для написания API
- Реляционная база данных
- Инструмент для сборки фронтенд-проекта

🔗 Шаг 03:

Вопрос 3

Сопоставьте ключевые элементы событийной архитектуры с их назначением.

Подсказка: Подумайте, кто создает событие, кто его передает и кто на него реагирует.

Ответ:

- **Producer** — Отправляет событие в систему
- **Consumer** — Подписывается и реагирует на события
- **Broker** — Хранит и передает события
- **Event** — Факт изменения, который передается

🔗 Шаг 04:

Вопрос 4

При разработке сервиса доставки программисты решили, что все вызовы будут через gRPC. Как gRPC помогает в событийно-ориентированной архитектуре?

Подсказка: Вспомните, чем отличается gRPC от брокера сообщений.

Ответ:

- **Обеспечивает быстрый и типобезопасный обмен сообщениями между сервисами**
- Хранит события до их обработки
- Заменяет Kafka как брокера
- Используется только для фронтенд-запросов

🔗 Шаг 05:

Вопрос 5

Вы строите пайплайн обработки транзакций. Расположите шаги работы событийного пайплайна в правильном порядке:

1. Сервис-потребитель сохраняет событие в Redis для кэширования
2. Пользователь инициирует действие (например, оплату)
3. Producer публикует событие в Kafka
4. Consumer обрабатывает событие

Подсказка: Начните с действия пользователя, подумайте, через кого идет событие, и кто завершает процесс.

Ответ:

- Пользователь инициирует действие (например, оплату)
- Producer публикует событие в Kafka
- Consumer обрабатывает событие
- Сервис-потребитель сохраняет событие в Redis для кэширования

🔗 Шаг 06:

Вопрос 6

У вас возникла ситуация: события начали накапливаться, а потребители обрабатывают их слишком медленно. Какой механизм Kafka помогает избежать перегрузки?

Подсказка: Вспомните про разделение нагрузки и параллелизм.

Ответ:

- **Партиции и Consumer Group для распределенной обработки**
- Автоматическое удаление событий при перегрузке
- Сжатие событий в один большой пакет
- Отключение логирования сообщений

🔗 Шаг 07:

Вопрос 7

Сопоставьте типы взаимодействия в распределенных системах.

Подсказка: Подумайте, чем отличается синхронный вызов от событийного подхода.

Ответ:

- **Синхронный запрос** — Отправитель ждёт немедленного ответа
- **Асинхронное сообщение** — Отправитель не ждёт, событие обрабатывается позже
- **Событийный обмен** — Сервис публикует событие, подписчики реагируют
- **Polling** — Сервис периодически спрашивает о новых данных

🔗 Шаг 08:

Вопрос 8

Разработчики интегрируют Redis в Event-driven пайплайн. Какую задачу Redis решает в этой архитектуре?

Подсказка: Вспомните, чем кэширование помогает при большом потоке событий.

Ответ:

- **Быстрое хранение и доступ к часто используемым данным**
- Замена брокера Kafka
- Создание API-интерфейсов
- Выполнение распределённых транзакций

🔗 Шаг 09:

Вопрос 9

У вас проект: при создании нового пользователя в системе нужно выполнить цепочку действий. Расположите шаги в правильном порядке:

1. Сервис регистрации публикует событие «UserCreated» в Kafka
2. Сервис email отправляет приветственное письмо
3. Пользователь регистрируется через форму
4. Сервис аналитики фиксирует метрику нового пользователя

Подсказка: Начало — действие пользователя, конец — аналитика.

Ответ:

- Пользователь регистрируется через форму
- Сервис регистрации публикует событие «UserCreated» в Kafka

- Сервис email отправляет приветственное письмо
- Сервис аналитики фиксирует метрику нового пользователя

Шаг 10:

Вопрос 10

Команда разработчиков планирует перейти от монолита к Event-driven архитектуре. Какое преимущество они получают в первую очередь?

Подсказка: Подумайте о масштабируемости и независимости сервисов.

Ответ:

- **Возможность масштабировать и развивать сервисы независимо**
- Все сервисы будут использовать одну большую базу данных
- Потребность писать меньше тестов
- Полное отсутствие ошибок в системе

Урок 2 - Сравнение requestresponse vs event-driven подхода

Шаг 01:

Вопрос 1

Сервис оплаты должен уведомить склад о новом заказе. Какое отличие между request/response и event-driven подходом проявится здесь?

Подсказка: Подумайте, ждёт ли сервис оплаты ответа от склада.

Ответ:

- **В event-driven подходе сервис оплаты публикует событие и не ждёт ответа**
- В обоих подходах сервисы обязательно ждут ответа
- В request/response подходе сервис оплаты работает асинхронно
- В event-driven подходе сервис оплаты всегда напрямую вызывает API склада

Шаг 02:

Вопрос 2

В интернет-магазине при создании заказа сервис логистики получает уведомление. Как проще всего описать поведение request/response архитектуры?

Подсказка: Вспомните, как происходит обмен: кто вызывает, а кто отвечает.

Ответ:

- **Сервис заказов напрямую вызывает API сервиса логистики и ждёт ответ**
- Сервис логистики публикует событие, а заказы реагируют
- Оба сервиса пишут в общую базу и читают из неё
- Ни один сервис не зависит от другого

Шаг 03:

Вопрос 3

Сопоставьте характеристики request/response и event-driven подхода.

Подсказка: Обратите внимание на зависимость сервисов и скорость реакции.

Ответ:

- **Request/Response** — Сервисы жёстко связаны и ожидают ответа
- **Event-driven** — Сервисы реагируют на события асинхронно

- **Высокая связанность** — Характерна для request/response
- **Низкая связанность** — Характерна для event-driven

Шаг 04:

Вопрос 4

В приложении доставки нужно отправить SMS пользователю после подтверждения заказа. Какое решение будет более гибким?

Подсказка: Вспомните, какой подход позволяет подключать новые сервисы без изменения существующих.

Ответ:

- **Использовать event-driven, где SMS-сервис подпишется на событие «OrderConfirmed»**
- Вызвать SMS-сервис напрямую из сервиса заказов через API
- Хранить все уведомления в одной таблице базы данных
- Создать cron-задачу, которая раз в час проверяет новые заказы

Шаг 05:

Вопрос 5

Представьте: пользователь оформляет заказ. Расположите шаги работы в request/response архитектуре в правильном порядке:

1. Сервис заказов вызывает API оплаты
2. Пользователь нажимает «Купить»
3. Сервис оплаты возвращает результат
4. Сервис заказов вызывает API склада

Подсказка: Начало — действие пользователя, затем идёт последовательная цепочка вызовов.

Ответ:

- Пользователь нажимает «Купить»
- Сервис заказов вызывает API оплаты
- Сервис оплаты возвращает результат
- Сервис заказов вызывает API склада

Шаг 06:

Вопрос 6

В системе event-driven пользователи часто жалуются на задержки обработки. Как программисту диагностировать проблему?

Подсказка: Подумайте о месте, где могут накапливаться события.

Ответ:

- **Проверить очередь брокера сообщений на наличие необработанных событий**
- Проверить количество API-запросов к базе данных
- Увеличить число cron-задач
- Поменять имена методов в коде

Шаг 07:

Вопрос 8

Разработчик выбирает между request/response и event-driven. Для какой задачи event-driven будет особенно полезен?

Подсказка: Подумайте о масштабируемости и возможности подключать новых потребителей.

Ответ:

- **Для рассылки уведомлений многим сервисам при одном событии**
- Для проверки пароля при входе
- Для подсчёта длины строки
- Для локального кеширования конфигурации

Шаг 08:

Вопрос 9

Сравните архитектуры: расположите шаги в event-driven пайплайне в правильном порядке:

1. Producer публикует событие
2. Consumer реагирует на событие
3. Пользователь совершает действие
4. Брокер сообщений сохраняет событие

Подсказка: Начало — пользователь, конец — реакция потребителя.

Ответ:

- Пользователь совершает действие
- Producer публикует событие
- Брокер сообщений сохраняет событие
- Consumer реагирует на событие

Шаг 09:

Вопрос 10

Компания переводит систему из request/response в event-driven. Какое преимущество они получают в первую очередь?

Подсказка: Вспомните про независимость и масштабируемость сервисов.

Ответ:

- **Сервисы смогут развиваться и масштабироваться независимо друг от друга**
- Все сервисы будут работать только синхронно
- Не придётся писать тесты
- Система станет полностью безошибочной

Урок 3 - Обзор технологий gRPC, Kafka, Redis

Шаг 01:

Вопрос 1

В команде появился новый сервис, и вам нужно решить, как он будет общаться с остальными. Выбор между request/response и event-driven стоит остро. Какой главный плюс event-driven?

Подсказка: Подумайте о слабой связанности и реакции на изменения.

Ответ:

- **Сервисы реагируют на события и остаются слабо связанными**
- Все сервисы делят одну базу данных
- Все запросы идут строго синхронно

- Каждый сервис знает внутренности других

🔔 Шаг 02:

Вопрос 2

В проекте используются разные технологии. Сопоставьте инструмент с его основной ролью.

🔔 Подсказка:

Вспомните: gRPC — про вызовы, Kafka — про сообщения, Redis — про хранение быстрых данных.

Ответ:

- **gRPC** — Эффективный способ удалённого вызова методов
- **Kafka** — Брокер событий для обмена сообщениями
- **Redis** — Быстрое in-мемори хранилище данных

🔔 Шаг 03:

Вопрос 3

При обработке заказов магазин хочет реагировать на события: «заказ создан», «заказ оплачен». Какой паттерн архитектуры это иллюстрирует?

Подсказка: Подумайте, что запускает реакцию других сервисов.

Ответ:

- **Event-driven архитектура**
- Монолитная архитектура
- Client-server без событий
- Схема общий доступ к файлам

🔔 Шаг 04:

Вопрос 4

Вы решаете, как сервисы будут взаимодействовать через Kafka. Расположите шаги по работе с событиями в правильном порядке.

🔔 Подсказка:

Ответ:

- Сначала сервис генерирует событие.
- Затем оно попадает в брокер.
- Другие сервисы его читают.
- И уже после выполняют свои действия.
- Сервис публикует событие
- Kafka сохраняет и распределяет его
- Сервисы-подписчики получают событие
- Подписчики выполняют бизнес-логику

🔔 Шаг 05:

Вопрос 5

Вам нужно построить быстрый кэш для результатов запросов, чтобы разгрузить базу. Что выбрать?

Подсказка: Подумайте про in-мемори хранилище.

Ответ:

- **Redis**
- **Kafka**
- **gRPC**

Шаг 06:

Вопрос 6

У вас несколько способов интеграции сервисов. Сопоставьте технологию с типичным сценарием её использования.

Подсказка:

Вспомните реальные задачи: быстрый кэш, надёжные очереди, вызов методов.

Ответ:

- **Kafka** — Обработка событий с гарантированной доставкой
- **Redis** — Хранение сессий пользователей
- **gRPC** — Вызов метода одного сервиса из другого

Шаг 07:

Вопрос 7

В вашей системе gRPC используется для вызовов между сервисами. Почему это эффективнее, чем обычный REST в JSON?

Подсказка: Подумайте о формате сериализации и скорости.

Ответ:

- **gRPC использует бинарный формат Protobuf, что быстрее и компактнее**
- gRPC всегда работает только офлайн
- REST не умеет работать с JSON
- gRPC заменяет Kafka и Redis

Шаг 08:

Вопрос 8

В системе заказов выстроен пайплайн: заказ создаётся, сообщение публикуется в Kafka, сервис оплаты реагирует, Redis хранит статус. Расположите шаги в правильной последовательности.

Подсказка:

Подумайте: сначала заказ, потом событие, потом обработка, потом запись статуса.

Ответ:

- Создание заказа пользователем
- Kafka получает событие
- Сервис оплаты проверяет оплату
- Redis обновляет статус заказа

Шаг 09:

Вопрос 9

Сопоставьте сценарии отказоустойчивости с тем, какая технология помогает их решить.

Подсказка:

Kafka сохраняет события, Redis даёт быстрый доступ к данным, gRPC обеспечивает надёжный вызов.

Ответ:

- **Kafka** — Не потерять события даже при падении сервиса
- **Redis** — Быстро восстановить данные сессии
- **gRPC** — Обеспечить стабильный удалённый вызов метода

 **Шаг 10:**

Вопрос 10

Вы спроектировали пайплайн: gRPC для синхронных вызовов, Kafka для событий, Redis для кэша. Какое главное преимущество такой комбинации?

Подсказка: Вспомните, что разные технологии решают разные задачи.

Ответ:

- **Каждый инструмент решает свою задачу, обеспечивая масштабируемый и надёжный пайплайн**
- Все технологии заменяют друг друга
- Пайплайн работает только на одном сервере
- Redis берёт на себя роль брокера сообщений

Урок 4 - Архитектурная схема проекта

 **Шаг 01:**

Вопрос 1

Что является ключевой особенностью event-driven архитектуры?

Подсказка: Подумайте, как системы узнают о событиях друг друга.

Ответ:

- **Компоненты реагируют на события асинхронно**
- Все компоненты вызываются синхронно через один основной поток
- Все данные обрабатываются только по расписанию
- Система использует только REST-запросы

 **Шаг 02:**

Вопрос 2

Сопоставьте компоненты event-driven архитектуры с их функцией.

Компоненты:

Ответ:

- Producer
- Consumer
- Event Broker
- Event

Функции:

1. Создает события и отправляет их в систему
2. Принимает события и выполняет действия
3. Посредник, передающий события между компонентами
4. Сообщение, которое информирует о произошедшем событии

 **Подсказка:**

Подумайте, кто инициирует событие, кто его обрабатывает, и кто обеспечивает доставку.

- **Producer** — Создает события и отправляет их в систему
- **Consumer** — Принимает события и выполняет действия
- **Event Broker** — Посредник, передающий события между компонентами
- **Event** — Сообщение, которое информирует о произошедшем событии

🔗 Шаг 03:

Вопрос 3

Как gRPC помогает в event-driven пайплайне?

Подсказка: Подумайте, как компоненты могут обмениваться данными быстро и структурировано.

Ответ:

- **Обеспечивает высокопроизводительный обмен данными между сервисами**
- Служит только для хранения событий
- Автоматически маршрутизирует события между очередями
- Заменяет Kafka и Redis в пайплайне

🔗 Шаг 04:

Вопрос 4: Расположите шаги обработки события в правильном порядке:

1. Consumer обрабатывает событие
2. Producer создаёт событие
3. Event Broker передает событие
4. Событие сохраняется в хранилище (например, Redis)

🔗 **Подсказка:**

Сначала событие создается, потом передается, обрабатывается и сохраняется.

Ответ:

- Producer создаёт событие
- Event Broker передает событие
- Consumer обрабатывает событие
- Событие сохраняется в хранилище (например, Redis)

🔗 Шаг 05:

Вопрос 5

Какая основная функция Kafka в event-driven пайплайне?

Подсказка: Подумайте о масштабируемой очереди событий.

Ответ:

- **Обеспечивает надёжную и масштабируемую очередь событий**
- Сохраняет только финальные результаты обработки
- Заменяет Redis для кэширования данных
- Автоматически создает gRPC-сервисы

🔗 Шаг 06:

📄 Вопрос 6

Сопоставьте тип события с подходящим обработчиком.

События:

Ответ:

- UserCreated
- OrderPaid
- InventoryUpdated
- NotificationSent

Обработчики:

1. Создает пользовательский профиль в системе
2. Обновляет остатки товаров на складе
3. Подтверждает оплату и инициирует доставку
4. Отправляет уведомление пользователю

 **Подсказка:**

Подумайте, что инициируется событием и какой сервис должен реагировать.

- **UserCreated** — Создает пользовательский профиль в системе
- **OrderPaid** — Подтверждает оплату и инициирует доставку
- **InventoryUpdated** — Обновляет остатки товаров на складе
- **NotificationSent** — Отправляет уведомление пользователю

 **Шаг 07:**

Вопрос 7

Для чего часто используют Redis в event-driven пайплайне?

Подсказка: Подумайте о быстром доступе к данным и временных хранилищах.

Ответ:

- **Кэширование событий и быстрый доступ к состоянию**
- Управление gRPC-соединениями
- Роутинг сообщений между сервисами
- Создание сложных отчётов

 **Шаг 08:**

Вопрос 8: Расположите шаги обработки события OrderPaid в правильной последовательности:

1. Сервис уведомлений отправляет email пользователю
2. Order сервис проверяет оплату
3. Inventory сервис обновляет остатки товаров
4. Event Broker передаёт событие всем подписчикам

 **Подсказка:**

Событие сначала создается, передается через брокер, а потом обрабатывается подписчиками в правильном порядке.

Ответ:

- Order сервис проверяет оплату
- Event Broker передаёт событие всем подписчикам
- Inventory сервис обновляет остатки товаров
- Сервис уведомлений отправляет email пользователю

Шаг 09:

Вопрос 9

Сопоставьте компонент с его ролью в пайплайне:

Компоненты:

Ответ:

- gRPC
- Kafka
- Redis

Роли:

1. Передача сообщений между сервисами
2. Быстрое хранение состояния и кэширование
3. Обмен данными между клиентом и сервером с низкой задержкой

Подсказка:

Подумайте, кто отвечает за транспорт, кто за очередь, а кто за кэш.

- **gRPC** — Обмен данными между клиентом и сервером с низкой задержкой
- **Kafka** — Передача сообщений между сервисами
- **Redis** — Быстрое хранение состояния и кэширование

Шаг 10:

Вопрос 10

Какой подход обеспечивает наибольшую надёжность и масштабируемость event-driven пайплайна с gRPC, Kafka и Redis?

Подсказка: Подумайте о разделении ответственности и асинхронной обработке.

Ответ:

- **Разделение сервисов, асинхронная обработка событий через Kafka, кэширование состояния в Redis и gRPC для обмена данными**
- Объединение всех функций в один монолитный сервис
- Использование только gRPC без брокера сообщений
- Синхронная обработка всех событий через один поток

Урок 5 - Практика разбор пайплайна заказов

Шаг 01:

Вопрос 1

Вы начали проект интернет-магазина. Вам нужно организовать обработку заказов так, чтобы каждое действие (создание заказа, оплата, доставка) было событием. Какой принцип лучше всего описывает этот подход?

Подсказка: Подумайте про архитектуру, где события запускают реакции в других сервисах.

Ответ:

- **Event-driven архитектура**
- Монолитная архитектура
- MVC-подход
- Микросервисы без событий

Шаг 02:

Вопрос 2

При создании заказа сервис должен отправить событие в Kafka, чтобы другие сервисы знали о новом заказе. Какую роль здесь выполняет Kafka?

Подсказка: Вспомните, кто выступает посредником между сервисами.

Ответ:

- **Брокер сообщений**
- База данных
- HTTP-сервер
- Балансировщик нагрузки

Шаг 03:

Вопрос 3

Сопоставьте элементы event-driven пайплайна с их описанием.

Подсказка: Вспомните, кто производит событие, кто его доставляет и кто его использует.

Ответ:

- **Producer** — Отправляет событие
- **Consumer** — Обработывает событие
- **Broker** — Хранит и доставляет события
- **Event** — Сообщение о произошедшем действии

Шаг 04:

Вопрос 4

Расположите шаги обработки нового заказа в правильном порядке.

Подсказка: Вспомните: сначала заказ создаётся, затем отправляется событие, после чего его обрабатывают другие сервисы.

Ответ:

- Создание заказа в сервисе
- Отправка события в Kafka
- Подписка сервисов на событие
- Обработка заказа другими сервисами

Шаг 05:

Вопрос 5

Сервис оплаты получил событие о новом заказе. Что он должен сделать в рамках event-driven подхода?

Подсказка: Вспомните, что сервис не меняет чужие данные напрямую, а реагирует своим событием.

Ответ:

- **Отправить событие об успешной или неуспешной оплате**
- Изменить заказ напрямую в чужой базе
- Игнорировать событие
- Удалить заказ

🔗 Шаг 06:

Вопрос 6

Сопоставьте использование технологий с их задачами.

Подсказка: gRPC — для общения сервисов напрямую, Kafka — для событий, Redis — для временного хранения.

Ответ:

- **gRPC** — Быстрый вызов методов между сервисами
- **Kafka** — Передача событий между сервисами
- **Redis** — Кэширование и хранение состояний

🔗 Шаг 07:

Вопрос 7

В сервисе доставки произошёл сбой, и он временно не может обрабатывать события. Как правильно действовать?

Подсказка: Вспомните про надёжность брокера сообщений.

Ответ:

- **События сохраняются в Kafka и будут доставлены, когда сервис восстановится**
- Все события будут потеряны
- Сервису придётся самому заново запрашивать все события у клиентов
- Нужно перезаписать события вручную в базу

🔗 Шаг 08:

Вопрос 8

Расположите шаги построения пайплайна заказов по сложности внедрения.

Подсказка: Сначала простая интеграция, потом добавление событий, потом кэширование.

Ответ:

- Прямой вызов gRPC для создания заказа
- Отправка события о заказе в Kafka
- Подписка сервисов оплаты и доставки на события
- Использование Redis для кэширования состояния заказа

🔗 Шаг 09:

Вопрос 9

Сопоставьте проблему с решением в event-driven пайплайне.

Подсказка: Обратите внимание на отказоустойчивость, скорость и асинхронность.

Ответ:

- **Высокая нагрузка на сервис** — Kafka распределяет сообщения по нескольким консюмерам
- **Долгие запросы между сервисами** — Использование асинхронных событий вместо синхронных вызовов
- **Частые одинаковые запросы** — Кэширование данных в Redis

🔗 Шаг 10:

Вопрос 10

Компания решила расширить пайплайн заказов: теперь нужно интегрировать аналитику, которая считает количество оплаченных заказов. Как лучше всего подключить сервис аналитики?

Подсказка: В event-driven архитектуре новые сервисы подключаются к событиям.

Ответ:

- **Подписать сервис аналитики на события об успешной оплате**
- Добавить вызовы аналитики прямо в сервис оплаты
- Сохранять данные в CSV и загружать вручную
- Периодически опрашивать базу заказов

Модуль 2 - gRPC API-сервис

Урок 1 - Протоколы взаимодействия (HTTPREST vs gRPC)

Шаг 01:

Вопрос 1

Что является основным преимуществом gRPC перед REST при коммуникации между микросервисами?

Подсказка: Подумайте о формате передачи данных и скорости.

Ответ:

- **Быстрая бинарная сериализация через Protocol Buffers**
- Использование только текстового JSON
- Нативная поддержка только браузерами
- Нет необходимости в схемах данных

Шаг 02:

Вопрос 2

Как gRPC обрабатывает потоковые данные между клиентом и сервером?

Подсказка: Обратите внимание на типы потоков: uni- и bidi-directional.

Ответ:

- **Поддерживает как односторонние, так и двусторонние потоки**
- Только односторонние потоки
- Только двусторонние потоки
- Не поддерживает потоковую передачу

Шаг 03:

Вопрос 3

Сопоставьте протокол с его характеристикой:

Протоколы:

Ответ:

- REST
- gRPC
- HTTP/1.1
- Protocol Buffers

Характеристики:

1. Использует текстовый формат JSON, легко читаемый
2. Бинарный протокол для сериализации данных, эффективен по скорости
3. Старый HTTP-протокол без поддержки multiplexing
4. Современный фреймворк для быстрого взаимодействия между сервисами

🔗 **Подсказка:**

Сопоставляйте протоколы с их ключевыми особенностями.

- **REST** — Использует текстовый формат JSON, легко читаемый
- **gRPC** — Современный фреймворк для быстрого взаимодействия между сервисами
- **HTTP/1.1** — Старый HTTP-протокол без поддержки multiplexing
- **Protocol Buffers** — Бинарный протокол для сериализации данных, эффективен по скорости

🔗 **Шаг 04:**

Вопрос 4

Какая особенность позволяет gRPC быть более эффективным при передаче большого количества микросервисных запросов?

Подсказка: Подумайте о соединениях и multiplexing.

Ответ:

- **Использование HTTP/2 с multiplexing на одном соединении**
- Использование только одного TCP-соединения без multiplexing
- Обязательная сериализация в JSON
- Отсутствие потоковой передачи данных

🔗 **Шаг 05:**

Вопрос 5 Расположите шаги для реализации gRPC-сервиса в правильном порядке:

1. Создание .proto файла с определением сервиса
2. Генерация серверного и клиентского кода через protoc
3. Реализация методов сервиса на сервере
4. Запуск сервера и тестирование с клиентом

🔗 **Подсказка:**

Сначала нужно описать контракт, затем сгенерировать код, реализовать и протестировать.

Ответ:

- Создание .proto файла с определением сервиса
- Генерация серверного и клиентского кода через protoc
- Реализация методов сервиса на сервере
- Запуск сервера и тестирование с клиентом

🔗 **Шаг 06:**

Вопрос 6

Какие типы вызовов поддерживает gRPC?

Подсказка: Подумайте про потоковую передачу данных.

Ответ:

- **Unary (одиночный запрос-ответ)**
- **Server streaming (сервер отправляет поток данных)**
- **Client streaming (клиент отправляет поток данных)**
- **Bidirectional streaming (двусторонний поток)**

 **Шаг 07:**

Вопрос 7

Сопоставьте термин с описанием:

Ответ:

- Unary RPC
- Server streaming RPC
- Client streaming RPC
- Bidirectional streaming RPC

1. Один запрос, один ответ
2. Сервер отправляет несколько сообщений в ответ на один запрос
3. Клиент отправляет несколько сообщений, сервер отвечает одним
4. Обмен потоками данных в обе стороны одновременно

 **Подсказка:**

Обратите внимание на количество сообщений, которые отправляются каждой стороной.

- **Unary RPC** — Один запрос, один ответ
- **Server streaming RPC** — Сервер отправляет несколько сообщений в ответ на один запрос
- **Client streaming RPC** — Клиент отправляет несколько сообщений, сервер отвечает одним
- **Bidirectional streaming RPC** — Обмен потоками данных в обе стороны одновременно

 **Шаг 08:**

Вопрос 8

Почему gRPC часто используют для микросервисов вместо REST?

Подсказка: Подумайте о нагрузке и скорости передачи данных.

Ответ:

- **Меньшая нагрузка сети и высокая производительность**
- Отсутствие необходимости в API контрактах
- Все браузеры поддерживают его нативно
- Нет необходимости в HTTP/2

Урок 2 - Создание gRPC-сервиса для приема заказов

 **Шаг 01:**

Вопрос 1

Какой основной протокол используется в gRPC для обмена данными?

Подсказка: Подумайте о том, что gRPC работает поверх HTTP/2.

Ответ:

- **HTTP/2**
- HTTP/1.1

- WebSocket
- TCP только

🔗 Шаг 02:

Вопрос 2

Для чего в gRPC используется файл .proto?

Подсказка: Подумайте о контракте между клиентом и сервером.

Ответ:

- **Для описания структуры сообщений и сервисов**
- Для хранения данных на сервере
- Для управления потоками TCP
- Для настройки Redis

🔗 Шаг 03:

Вопрос 3

Сопоставьте термин с его объяснением:

Ответ:

- Unary RPC
- Server Streaming RPC
- Client Streaming RPC
- Bidirectional Streaming RPC

1. Один запрос, один ответ
2. Сервер отправляет несколько сообщений клиенту
3. Клиент отправляет несколько сообщений серверу
4. Обмен потоками данных в обе стороны одновременно

🔗 **Подсказка:**

Обратите внимание, кто инициирует поток сообщений.

- **Unary RPC** — Один запрос, один ответ
- **Server Streaming RPC** — Сервер отправляет несколько сообщений клиенту
- **Client Streaming RPC** — Клиент отправляет несколько сообщений серверу
- **Bidirectional Streaming RPC** — Обмен потоками данных в обе стороны одновременно

🔗 Шаг 04:

Вопрос 4

Что необходимо сделать после создания .proto файла для gRPC сервиса?

Подсказка: Подумайте о генерации кода.

Ответ:

- **Сгенерировать клиентский и серверный код с помощью protoc**
- Сразу запускать сервер без генерации
- Сохранять только в Redis
- Написать HTML страницу для клиента

🔗 Шаг 05:

Вопрос 5 Расположите шаги по созданию gRPC-сервиса для приема заказов в правильном порядке:

1. Создать .proto файл с описанием сервиса и сообщений
2. Сгенерировать код клиента и сервера
3. Реализовать методы сервиса на сервере
4. Запустить сервер и протестировать клиентом

🔗 Подсказка:

Сначала контракт, затем генерация, реализация и тестирование.

Ответ:

- Создать .proto файл с описанием сервиса и сообщений
- Сгенерировать код клиента и сервера
- Реализовать методы сервиса на сервере
- Запустить сервер и протестировать клиентом

🔗 Шаг 06:

Вопрос 6

Какой формат данных gRPC использует для передачи сообщений?

Подсказка: Подумайте о бинарной сериализации.

Ответ:

- **Protocol Buffers (Protobuf)**
- JSON
- XML
- CSV

🔗 Шаг 07:

Вопрос 7

Сопоставьте понятие с описанием:

Ответ:

- Client Stub
 - Server Skeleton
 - RPC
 - gRPC Channel
1. Локальный интерфейс для вызова удаленных методов сервера
 2. Интерфейс на сервере для обработки вызовов от клиента
 3. Удаленный вызов метода на сервере
 4. Соединение для передачи сообщений между клиентом и сервером

🔗 Подсказка:

Подумайте о роли каждого компонента в коммуникации.

- **Client Stub** — Локальный интерфейс для вызова удаленных методов сервера
- **Server Skeleton** — Интерфейс на сервере для обработки вызовов от клиента
- **RPC** — Удаленный вызов метода на сервере
- **gRPC Channel** — Соединение для передачи сообщений между клиентом и сервером

Урок 3 - Определение protobuf-схем

Шаг 01:

Вопрос 1

Какой язык используется для описания сообщений и сервисов в gRPC?

Подсказка: Подумайте о формате, который компилируется в код разных языков.

Ответ:

- **Protocol Buffers (.proto)**
- JSON Schema
- XML Schema
- YAML

Шаг 02:

Вопрос 2

Что задается в поле message .proto файла?

Подсказка: Подумайте о структуре данных, которую отправляет клиент или сервер.

Ответ:

- **Определение структуры данных с полями и типами**
- Определение методов сервера
- Настройка Kafka топиков
- Хранение данных в Redis

Шаг 03:

Вопрос 3

Какие типы данных поддерживаются в Protobuf?

Подсказка: Обратите внимание на целые, строковые и логические значения.

Ответ:

- **int32**
- **string**
- **bool**
- HTML
- CSS
- XML

Шаг 04:

Вопрос 4

Сопоставьте элементы protobuf с их описанием:

Ответ:

- service
- rpc
- message
- field

1. Определяет сервис, доступный клиенту
2. Определяет метод сервиса

3. Определяет структуру данных
4. Определяет поле внутри сообщения с типом и номером

🔔 **Подсказка:**

Подумайте о том, что описывает данные и что методы.

- **service** — Определяет сервис, доступный клиенту
- **rpc** — Определяет метод сервиса
- **message** — Определяет структуру данных
- **field** — Определяет поле внутри сообщения с типом и номером

🔔 **Шаг 05:**

Вопрос 5

Что означает номер поля в message Protobuf?

Подсказка: Подумайте о том, как сериализуются данные для передачи по сети.

Ответ:

- **Уникальный идентификатор поля для сериализации**
- Номер версии сервера
- Номер Kafka топика
- Номер Redis ключа

🔔 **Шаг 06:**

Вопрос 6 Расположите шаги по созданию protobuf-схемы для нового сервиса:

1. Определить структуру сообщений (message)
2. Определить сервис (service) и методы (rpc)
3. Назначить уникальные номера полям
4. Сгенерировать код клиента и сервера с помощью protoc

🔔 **Подсказка:**

Сначала определяем данные, затем сервис и поля, и только потом генерация кода.

Ответ:

- Определить структуру сообщений (message)
- Определить сервис (service) и методы (rpc)
- Назначить уникальные номера полям
- Сгенерировать код клиента и сервера с помощью protoc

🔔 **Шаг 07:**

Вопрос 7

Как можно определить повторяющееся поле (массив) в Protobuf?

Подсказка: Используется ключевое слово для коллекций.

Ответ:

- **repeated**
- array
- list
- map

🔧 Шаг 08:

Вопрос 8

Сопоставьте типы данных protobuf с примерами:

Ответ:

- int32
 - string
 - bool
 - double
1. Целое число 32-бита
 2. Строка текста
 3. Логическое значение true/false
 4. Число с плавающей точкой двойной точности

🔧 Подсказка:

Обратите внимание на тип данных и пример значения.

- **int32** — Целое число 32-бита
- **string** — Строка текста
- **bool** — Логическое значение true/false
- **double** — Число с плавающей точкой двойной точности

🔧 Шаг 09:

Вопрос 9

Что нужно учитывать при изменении существующей protobuf-схемы?

Подсказка: Подумайте о совместимости клиентов и серверов.

Ответ:

- **Не удалять существующие номера полей, добавлять новые с новыми номерами**
- Удалять все старые поля и перенумеровывать
- Менять тип поля с int32 на string без предупреждения
- Удалять service и создавать новый

🔧 Шаг 10:

Вопрос 10

Почему использование protobuf важно для event-driven пайплайна на gRPC, Kafka и Redis?

Подсказка: Подумайте о скорости сериализации и компактности данных.

Ответ:

- **Быстрая сериализация и компактный бинарный формат, удобный для передачи сообщений**
- Для хранения данных в XML
- Чтобы заменить gRPC
- Для генерации CSS

📖 Урок 4 - Генерация клиентских и серверных заглушек

🔧 Шаг 01:

Вопрос 1

Вы работаете над новым gRPC-сервисом и хотите быстро протестировать клиентскую часть. Что нужно сделать в первую очередь?

Подсказка: Подумайте о том, что обычно генерируется из .proto файлов для клиента.

Ответ:

- **Сгенерировать клиентские заглушки с помощью protoc**
- Написать все методы вручную
- Создать Kafka топики
- Запустить Redis сервер

🔮 Шаг 02:

Вопрос 2

Для чего нужны серверные заглушки в gRPC?

Подсказка: Они помогают тестировать клиент без полной реализации сервера.

Ответ:

- **Для имитации поведения сервера при тестировании клиента**
- Для генерации Kafka сообщений
- Для хранения данных в Redis
- Для написания фронтенда

🔮 Шаг 03:

Вопрос 3

Сопоставьте тип заглушки с её назначением:

Ответ:

- Клиентская заглушка
 - Серверная заглушка
 - Интерфейс gRPC
 - Protobuf message
1. Имитация сервера для тестирования клиента
 2. Генерация клиентского кода для вызовов методов
 3. Определяет методы и сервисы
 4. Определяет структуру передаваемых данных

🔮 Подсказка:

Подумайте, что генерируется для клиента и для сервера.

- **Клиентская заглушка** — Генерация клиентского кода для вызовов методов
- **Серверная заглушка** — Имитация сервера для тестирования клиента
- **Интерфейс gRPC** — Определяет методы и сервисы
- **Protobuf message** — Определяет структуру передаваемых данных

🔮 Шаг 04:

Вопрос 4

Вы обновили .proto файл. Что нужно сделать с заглушками, чтобы изменения применились?

Подсказка: Заглушки не обновятся сами.

Ответ:

- **Перегенерировать клиентские и серверные заглушки**

- Перезапустить Kafka
- Удалить Redis ключи
- Ничего не делать

🔗 Шаг 05:

Вопрос 5

Какие команды обычно используются для генерации gRPC-заглушек на Go?

Подсказка: Обратите внимание на protoc и плагины.

Ответ:

- **protoc --go_out=. --go-grpc_out=. service.proto**
- **Использование плагина go-grpc**
- npm install grpc-tools
- docker build .

🔗 Шаг 06:

Вопрос 6 Расположите шаги тестирования клиента с серверной заглушкой:

1. Сгенерировать серверную заглушку
2. Запустить заглушку локально
3. Сгенерировать клиентские заглушки
4. Написать тестовые вызовы методов клиента

🔗 **Подсказка:**

Сначала сервер, потом клиент, потом тесты.

Ответ:

- Сгенерировать серверную заглушку
- Запустить заглушку локально
- Сгенерировать клиентские заглушки
- Написать тестовые вызовы методов клиента

🔗 Шаг 07:

Вопрос 7

При тестировании пайплайна с Kafka и gRPC клиентскими заглушками возникает проблема: клиент не получает сообщения. Что проверить в первую очередь?

Подсказка: Подумайте о подключении к брокеру и генерации методов.

Ответ:

- **Проверить, что заглушка корректно подключается к серверу и методы сгенерированы**
- Проверить наличие Docker контейнеров Redis
- Проверить файл .env
- Регенерировать Kafka топик

🔗 Шаг 08:

Вопрос 8

Сопоставьте компоненты с их ролью при генерации заглушек:

Ответ:

- protoc
 - go-grpc плагин
 - Клиентская заглушка
 - Серверная заглушка
1. Компилирует .proto файл в код
 2. Генерирует gRPC методы для Go
 3. Имитация сервера для тестирования клиента
 4. Генерация кода для вызова методов сервера

 **Подсказка:**

Разделите задачи компиляции и генерации.

- **protoc** — Компилирует .proto файл в код
- **go-grpc плагин** — Генерирует gRPC методы для Go
- **Клиентская заглушка** — Генерация кода для вызова методов сервера
- **Серверная заглушка** — Имитация сервера для тестирования клиента

 **Шаг 09:**

Вопрос 9

Почему важно регенерировать заглушки после изменения .proto файлов?

Подсказка: Подумайте о том, чтобы клиент и сервер понимали одинаковую структуру данных.

Ответ:

- **Чтобы изменения в сообщениях и методах были отражены в коде клиента и сервера**
- Чтобы пересобрать Kafka топики
- Чтобы обновить Redis ключи
- Чтобы перезапустить сервер

 **Шаг 10:**

Вопрос 10

Как генерация клиентских и серверных заглушек ускоряет разработку event-driven пайплайна с gRPC, Kafka и Redis?

Подсказка: Подумайте о тестировании и независимой разработке компонентов.

Ответ:

- **Позволяет параллельно разрабатывать и тестировать клиент и сервер без полной реализации**
- Заменяет Kafka и Redis
- Позволяет писать SQL-запросы
- Автоматически создает фронтенд

Урок 5 - Практика написать gRPC API, принимающий заказ от клиента.

 **Шаг 01:**

Вопрос 1

Какой метод в gRPC автоматически создается на клиентской стороне после компиляции .proto файла?

Подсказка: Он позволяет клиенту вызывать удалённые методы как обычные функции.

Ответ:

- **Клиентская заглушка (Client Stub)**
- Серверная заглушка (Server Stub)
- gRPC middleware

🔮 Шаг 02:

Вопрос 2

Что делает серверная заглушка в gRPC?

Подсказка: Подумайте, где программист пишет логику обработки запросов.

Ответ:

- **Определяет интерфейс методов, который сервер должен реализовать**
- Обработывает все клиентские подключения автоматически
- Запускает тесты gRPC API

🔮 Шаг 03:

Вопрос 3

Сопоставьте термины с их описанием:

Ответ:

- Клиентская заглушка
 - Серверная заглушка
 - gRPC API
1. Интерфейс для вызова удалённых методов
 2. Интерфейс для реализации сервера
 3. Сервис, предоставляющий удалённые методы

Подсказка: Думайте, кто инициирует вызов, а кто обрабатывает.

- **Клиентская заглушка** — Интерфейс для вызова удалённых методов
- **Серверная заглушка** — Интерфейс для реализации сервера
- **gRPC API** — Сервис, предоставляющий удалённые методы

🔮 Шаг 04:

Вопрос 4

Для чего используют bufconn при тестировании gRPC клиента?

Подсказка: Подумайте, как можно обойти настоящую сеть.

Ответ:

- **Имитация сетевого соединения для тестов без реального сервера**
- Запуск продакшн сервера локально
- Буферизация сообщений Kafka

🔮 Шаг 05:

Вопрос 5 Расположите шаги создания gRPC API сервиса:

1. Составление .proto файла с методами и сообщениями
2. Генерация клиентской и серверной заглушки

3. Реализация методов на сервере
4. Запуск сервера и проверка с клиентом

Подсказка: Начинаем с определения контрактов, потом реализация, потом тестирование.

Ответ:

- Составление .proto файла с методами и сообщениями
- Генерация клиентской и серверной заглушки
- Реализация методов на сервере
- Запуск сервера и проверка с клиентом

Шаг 06:

Вопрос 6

Какой тип gRPC метода подходит для передачи заказов от клиента в реальном времени?

Подсказка: Клиент может отправлять несколько сообщений и получать ответы по мере обработки.

Ответ:

- **Client Streaming или Bidirectional Streaming**
- Unary RPC
- Unary Blocking

Шаг 07:

Вопрос 7

Сопоставьте типы gRPC вызовов с их поведением:

Ответ:

- Unary
- Server Streaming
- Client Streaming
- Bidirectional Streaming

1. **Один запрос** — один ответ
2. Клиент отправляет поток сообщений, сервер один ответ
3. Клиент один запрос, сервер поток ответов
4. Оба обмениваются потоками сообщений

Подсказка: Обратите внимание, кто иницирует поток сообщений.

- **Unary** — Один запрос — один ответ
- **Server Streaming** — Клиент один запрос, сервер поток ответов
- **Client Streaming** — Клиент отправляет поток сообщений, сервер один ответ
- **Bidirectional Streaming** — Оба обмениваются потоками сообщений

Шаг 08:

Вопрос 9 Расположите этапы отладки gRPC сервиса с логированием:

1. Добавление логов на сервере и клиенте
2. Запуск тестового клиента и сервера
3. Сбор и анализ логов
4. Исправление ошибок и повторное тестирование

Подсказка: Логи помогают понять поток сообщений и выявить ошибки.

Ответ:

- Добавление логов на сервере и клиенте
- Запуск тестового клиента и сервера
- Сбор и анализ логов
- Исправление ошибок и повторное тестирование

💡 **Шаг 09:**

Вопрос 10

Какой инструмент помогает визуально мониторить gRPC метрики на продакшн-сервере?

Подсказка: Обычно интегрируется с Prometheus и Grafana.

Ответ:

- **Prometheus + Grafana**
- bufconn
- Mock client

📁 **Модуль 3 - Kafka Publisher**

📖 **Урок 1 - Устройство Kafka брокеры, топики, партиции**

💡 **Шаг 01:**

Вопрос 1

Что из перечисленного является компонентом Kafka?

Подсказка: Подумайте о том, что обрабатывает и хранит сообщения.

Ответ:

- **Брокер**
- Модель MVC
- DNS-сервер
- Компилятор Go

💡 **Шаг 02:**

Вопрос 2

Что такое топик в Kafka?

Подсказка: Это место, куда публикуются сообщения.

Ответ:

- **Логическая категория для сообщений**
- Файл конфигурации брокера
- Процесс gRPC-сервера
- Тип базы данных Redis

💡 **Шаг 03:**

Вопрос 3

Сопоставьте термин Kafka с его определением:

Термины:

Ответ:

- Брокер
- Топик
- Партиция

Определения:

1. Сервер Kafka, который принимает и хранит сообщения.
2. Логическая категория сообщений.
3. Подраздел топика для масштабирования и параллельной обработки.

 Подсказка:

Подумайте, как сообщения структурированы для распределения нагрузки.

- **Брокер** — Сервер Kafka, который принимает и хранит сообщения.
- **Топик** — Логическая категория сообщений.
- **Партиция** — Подраздел топика для масштабирования и параллельной обработки.

 Шаг 04:

Вопрос 4 Расставьте шаги публикации сообщения в Kafka в правильном порядке:

1. Сообщение отправляется в продюсер
2. Продюсер отправляет сообщение на брокер
3. Брокер записывает сообщение в партицию топика
4. Потребитель считывает сообщение из партиции

 Подсказка:

Начинается с продюсера, заканчивается на потребителе.

Ответ:

- Сообщение отправляется в продюсер
- Продюсер отправляет сообщение на брокер
- Брокер записывает сообщение в партицию топика
- Потребитель считывает сообщение из партиции

 Шаг 05:

Вопрос 5

Какой компонент Kafka гарантирует устойчивость сообщений?

Подсказка: Сообщения не теряются даже при сбое сервера.

Ответ:

- **Партиция с репликацией**
- Топик без партиций
- Потребитель
- gRPC-сервер

 Шаг 06:

Вопрос 6

Сопоставьте термин Kafka Producer с его функцией:

Ответ:

- Producer
 - Consumer
 - Partition
 - Broker
1. Отправка сообщений в Kafka
 2. Чтение сообщений из топика
 3. Деление топика на части
 4. Хранение и распределение сообщений

 Подсказка:

Подумайте, кто создает и кто читает данные.

- **Producer** — Отправка сообщений в Kafka
- **Consumer** — Чтение сообщений из топика
- **Partition** — Деление топика на части
- **Broker** — Хранение и распределение сообщений

 Шаг 07:

Вопрос 7

Какой способ гарантирует порядок сообщений в топике Kafka?

Подсказка: Важен порядок внутри одной партиции.

Ответ:

- **Отправка сообщений в одну партицию**
- Отправка сообщений в разные топика
- Использование нескольких продюсеров без ключа
- Чтение сообщений случайным образом

 Шаг 08:

Вопрос 8 Расставьте шаги настройки продюсера Kafka:

1. Указать адрес брокеров
2. Определить сериализацию сообщений
3. Создать объект продюсера
4. Отправлять сообщения

 Подсказка:

Сначала настраиваем соединение и формат данных, потом создаем продюсера и используем его.

Ответ:

- Указать адрес брокеров
- Определить сериализацию сообщений
- Создать объект продюсера
- Отправлять сообщения

 Шаг 09:

Вопрос 9

Что происходит, если продюсер отправляет сообщение с ключом в Kafka?

Подсказка: Ключ влияет на распределение по партициям.

Ответ:

- **Сообщение попадает в определённую партицию на основе ключа**
- Сообщение теряется при сбое брокера
- Сообщение автоматически дублируется в Redis
- Партиция выбирается случайным образом без учёта ключа

🔮 Шаг 10:

Вопрос 10

Сопоставьте сценарий с использованием Kafka:

Ответ:

- События заказов
 - Логи сервера
 - Обновления пользовательского профиля
 - Отправка уведомлений
1. Публикация сообщений в топик orders
 2. Поток логов в отдельный топик logs
 3. Обновление партиций топика user_profiles
 4. Отправка сообщений в топик notifications

🔮 Подсказка:

Подумайте, какие типы данных логически сгруппированы для Kafka.

- **События заказов** — Публикация сообщений в топик orders
- **Логи сервера** — Поток логов в отдельный топик logs
- **Обновления пользовательского профиля** — Обновление партиций топика user_profiles
- **Отправка уведомлений** — Отправка сообщений в топик notifications

📖 Урок 2 - Конфигурация Kafka и запуск в Docker

🔮 Шаг 01:

Вопрос 1

Какой образ Docker обычно используется для запуска Kafka?

Подсказка: Он часто используется вместе с Zookeeper.

Ответ:

- **confluentinc/cp-kafka**
- postgres:latest
- nginx:alpine
- redis:latest

🔮 Шаг 02:

Вопрос 2

Какой сервис обычно требуется вместе с Kafka для управления метаданными брокеров?

Подсказка: Он обеспечивает координацию и хранение состояния.

Ответ:

- **Zookeeper**
- Redis
- PostgreSQL
- Etcd

Шаг 03:

Вопрос 3

Сопоставьте конфигурационный параметр Kafka с его назначением:

Ответ:

- KAFKA_BROKER_ID
 - KAFKA_ADVERTISED_LISTENERS
 - KAFKA_ZOOKEEPER_CONNECT
1. Уникальный идентификатор брокера
 2. Адреса, по которым брокер доступен клиентам
 3. Адреса Zookeeper для подключения

Подсказка:

Подумайте, какие параметры нужны для идентификации, подключения к клиентам и координации.

- **KAFKA_BROKER_ID** — Уникальный идентификатор брокера
- **KAFKA_ADVERTISED_LISTENERS** — Адреса, по которым брокер доступен клиентам
- **KAFKA_ZOOKEEPER_CONNECT** — Адреса Zookeeper для подключения

Шаг 04:

Вопрос 4

Для чего используется параметр KAFKA_LOG_RETENTION_HOURS?

Подсказка: Подумайте о времени хранения сообщений в топиках.

Ответ:

- **Задание времени хранения сообщений в топике**
- Количество брокеров в кластере
- Количество потребителей
- Порт для подключения клиента

Шаг 05:

Вопрос 5 Расположите шаги запуска Kafka в Docker в правильном порядке:

1. Создание docker-compose.yml с конфигурацией Kafka и Zookeeper
2. Запуск контейнеров с помощью docker-compose up
3. Проверка состояния брокеров
4. Создание топиков для продюсеров и потребителей

Подсказка:

Сначала готовим конфигурацию, потом запускаем сервисы и проверяем их состояние, после чего используем топик.

Ответ:

- Создание docker-compose.yml с конфигурацией Kafka и Zookeeper
- Запуск контейнеров с помощью docker-compose up
- Проверка состояния брокеров
- Создание топиков для продюсеров и потребителей

 Шаг 06:

Вопрос 6

Какой способ проверки, что Kafka брокер запущен в Docker, является корректным?

Подсказка: Используйте инструменты для работы с топиками.

Ответ:

- **Использовать kafka-topics.sh --list**
- Открыть веб-браузер на localhost:80
- ping на порт 22 контейнера
- docker logs nginx

 Шаг 07:

Вопрос 7

Сопоставьте команды Kafka с их назначением:

Ответ:

- kafka-topics.sh --create
 - kafka-topics.sh --list
 - kafka-console-producer.sh
 - kafka-console-consumer.sh
1. Создание нового топика
 2. Список всех топиков
 3. Отправка сообщений в топик
 4. Чтение сообщений из топика

 Подсказка:

Подумайте, какая команда отвечает за создание, просмотр, публикацию и потребление сообщений.

- **kafka-topics.sh --create** — Создание нового топика
- **kafka-topics.sh --list** — Список всех топиков
- **kafka-console-producer.sh** — Отправка сообщений в топик
- **kafka-console-consumer.sh** — Чтение сообщений из топика

 Шаг 08:

Вопрос 8

Какой параметр docker-compose.yml определяет имя контейнера Kafka?

Подсказка: Смотрим на блок services и container_name.

Ответ:

- **container_name**
- image
- ports

- volumes

Шаг 09:

Вопрос 9 Расставьте шаги публикации сообщения с Kafka Producer после запуска контейнера:

1. Создание объекта Producer в коде
2. Настройка адреса брокеров и сериализации
3. Отправка сообщения в топик
4. Закрытие Producer после использования

Подсказка:

Сначала настраиваем соединение, потом создаем объект, используем его и закрываем.

Ответ:

- Создание объекта Producer в коде
- Настройка адреса брокеров и сериализации
- Отправка сообщения в топик
- Закрытие Producer после использования

Шаг 10:

Вопрос 10

Что произойдет, если контейнер Kafka не подключен к Zookeeper?

Подсказка: Kafka использует Zookeeper для управления метаданными кластера.

Ответ:

- **Брокер не сможет корректно запуститься**
- Сообщения будут храниться в Redis
- Топики создаются автоматически без проблем
- Docker сам исправит подключение

Урок 3 - Реализация Publisher-сервиса

Шаг 01:

Вопрос 1

Какой основной метод используется для отправки сообщений в Kafka с помощью Producer?

Подсказка: Он принимает объект сообщения и топик.

Ответ:

- **send**
- receive
- poll
- commit

Шаг 02:

Вопрос 2

Какой объект в Kafka Producer используется для настройки адреса брокеров и сериализации сообщений?

Подсказка: Обычно создается перед инициализацией Producer.

Ответ:

- **Properties / Config**
- ConsumerRecord
- TopicPartition
- ZookeeperClient

🔗 Шаг 03:

Вопрос 3 Расположите шаги отправки сообщения через Kafka Producer:

1. Создание объекта Producer
2. Настройка брокеров и сериализации
3. Формирование ProducerRecord с топиком и сообщением
4. Вызов метода send
5. Закрытие Producer

🔗 Подсказка:

Сначала конфигурируем Producer, потом создаем запись и отправляем её, затем закрываем объект.

Ответ:

- Создание объекта Producer
- Настройка брокеров и сериализации
- Формирование ProducerRecord с топиком и сообщением
- Вызов метода send
- Закрытие Producer

🔗 Шаг 04:

Вопрос 4

Что произойдет, если вызвать send без вызова close для Producer?

Подсказка: Producer держит буферы и соединения открытыми.

Ответ:

- **Сообщения могут не быть полностью отправлены**
- Сообщения автоматически сохраняются в Redis
- Producer автоматически закрывается
- Произойдет немедленный сброс всех сообщений в топик

🔗 Шаг 05:

Вопрос 5

Сопоставьте объекты Kafka Producer с их назначением:

Ответ:

- Producer
 - ProducerRecord
 - Serializer
1. Отправка сообщений в топик
 2. Описание сообщения с ключом, значением и топиком
 3. Преобразование объекта в байты для передачи

🔔 Подсказка:

Подумайте, что делает каждый объект: кто отправляет, кто формирует запись, кто сериализует.

- **Producer** — Отправка сообщений в топик
- **ProducerRecord** — Описание сообщения с ключом, значением и топиком
- **Serializer** — Преобразование объекта в байты для передачи

🔔 Шаг 06:

Вопрос 6

Что необходимо указать при создании ProducerRecord?

Подсказка: Топик и данные — обязательны для отправки.

Ответ:

- **Топик и сообщение (ключ опционально)**
- Только ключ сообщения
- Адрес Zookeeper
- Имя контейнера Docker

🔔 Шаг 07:

Вопрос 7 Расположите шаги обработки исключений при отправке сообщений Producer:

1. Попытка отправки сообщения
2. Логирование ошибки при исключении
3. Повторная отправка или откат
4. Закрытие Producer

🔔 Подсказка:

Сначала пробуем отправить, ловим исключения, логируем и предпринимаем действия по восстановлению.

Ответ:

- Попытка отправки сообщения
- Логирование ошибки при исключении
- Повторная отправка или откат
- Закрытие Producer

🔔 Шаг 08:

Вопрос 8

Что нужно настроить, чтобы Kafka Producer был готов к работе в многопоточном приложении?

Подсказка: Подумайте о безопасности потоков и повторных попытках.

Ответ:

- **enable.idempotence=true**
- **auto.offset.reset=earliest**
- **group.id**
- **zookeeper.connect**

🔔 Шаг 09:

Вопрос 9

Сопоставьте типы сериализации Kafka с примером:

Ответ:

- StringSerializer
- IntegerSerializer
- ByteArraySerializer

1. Преобразует строку в байты
2. Преобразует целое число в байты
3. Используется для отправки массива байт

 **Подсказка:**

Подумайте, какой сериализатор нужен для строк, чисел и сырого массива байт.

- **StringSerializer** — Преобразует строку в байты
- **IntegerSerializer** — Преобразует целое число в байты
- **ByteArraySerializer** — Используется для отправки массива байт

 **Шаг 10:**

Вопрос 10

Что произойдет, если Producer отправит сообщение в несуществующий топик?

Подсказка: Kafka может создать топик автоматически при соответствующей настройке.

Ответ:

- **Сообщение может быть отправлено, если включена авто-creation топика**
- Произойдет сбой Producer
- Сообщение сохраняется локально в Producer
- Kafka автоматически создаст Redis запись

Урок 4 - Публикация события «новый заказ» в Kafka

 **Шаг 01:**

Вопрос 1

Какой метод Kafka Producer используется для отправки события о новом заказе?

Подсказка: Он принимает объект ProducerRecord и возвращает Future.

Ответ:

- **send**
- receive
- commit
- poll

 **Шаг 02:**

Вопрос 2

Что обязательно нужно указать при создании события нового заказа для Kafka?

Подсказка: Без этих данных сообщение не будет доставлено в нужный топик.

Ответ:

- **Топик и объект заказа (ключ опционально)**
- Только ключ заказа
- Адрес Redis
- ID пользователя без данных заказа

🔗 Шаг 03:

Вопрос 3 Расположите шаги публикации нового заказа в Kafka:

1. Создать объект Producer
2. Настроить брокеры и сериализацию
3. Создать ProducerRecord с данными заказа
4. Вызвать send
5. Закрыть Producer

🔗 Подсказка:

Сначала готовим Producer, затем формируем сообщение и отправляем его, после чего закрываем соединение.

Ответ:

- Создать объект Producer
- Настроить брокеры и сериализацию
- Создать ProducerRecord с данными заказа
- Вызвать send
- Закрыть Producer

🔗 Шаг 04:

Вопрос 4

Что будет, если Producer не закрыть после отправки события о новом заказе?

Подсказка: Producer хранит буферы и открытые соединения.

Ответ:

- **Некоторые события могут не отправиться**
- События автоматически сохраняются в Redis
- Producer закроется автоматически
- Сообщения будут отсланы дважды

🔗 Шаг 05:

Вопрос 5

Сопоставьте объект Kafka с его ролью при публикации нового заказа:

Ответ:

- Producer
 - ProducerRecord
 - Serializer
1. Отправка событий в топик
 2. Описание заказа с ключом и значением
 3. Преобразование объекта заказа в байты

🔗 Подсказка:

Подумайте, кто отвечает за отправку, кто формирует запись, кто сериализует.

- **Producer** — Отправка событий в топик
- **ProducerRecord** — Описание заказа с ключом и значением
- **Serializer** — Преобразование объекта заказа в байты

🔗 Шаг 06:

Вопрос 6

Какая настройка Producer важна для гарантированной доставки события о заказе?

Подсказка: Подумайте о надежности и подтверждениях от брокеров.

Ответ:

- **acks=all**
- enable.auto.commit=false
- group.id
- auto.offset.reset=latest

🔗 Шаг 07:

Вопрос 7 Расположите действия при ошибке отправки события нового заказа:

1. Попытка отправки
2. Логирование ошибки
3. Повторная отправка или откат
4. Закрытие Producer

🔗 **Подсказка:**

Сначала пробуем отправить, при ошибке логируем и предпринимаем действия для восстановления.

Ответ:

- Попытка отправки
- Логирование ошибки
- Повторная отправка или откат
- Закрытие Producer

🔗 Шаг 08:

Вопрос 9

Сопоставьте типы сериализации Kafka с примером:

Ответ:

- StringSerializer
 - IntegerSerializer
 - ByteArraySerializer
1. Преобразует строку в байты
 2. Преобразует целое число в байты
 3. Используется для отправки массива байт

🔗 **Подсказка:**

Подумайте, какой сериализатор нужен для строк, чисел и сырого массива байт.

- **StringSerializer** — Преобразует строку в байты
- **IntegerSerializer** — Преобразует целое число в байты
- **ByteArraySerializer** — Используется для отправки массива байт

🔗 Шаг 09:

Вопрос 10

Что произойдет, если Producer отправит сообщение в несуществующий топик?

Подсказка: Kafka может создать топик автоматически при соответствующей настройке.

Ответ:

- **Сообщение может быть отправлено, если включена авто-creation топика**
- Произойдет сбой Producer
- Сообщение сохраняется локально в Producer
- Kafka автоматически создаст Redis запись

Урок 5 - Практика отправка заказов в Kafka из API-сервиса.

Шаг 01:

Вопрос 1

Какой компонент API-сервиса отвечает за отправку события о новом заказе в Kafka?

Подсказка: Он создаёт и конфигурирует Producer для публикации сообщений.

Ответ:

- **Kafka Producer**
- Kafka Consumer
- REST Controller
- Database Repository

Шаг 02:

Вопрос 2

Что нужно обязательно указать при формировании события нового заказа?

Подсказка: Сообщение не попадёт в Kafka без этих данных.

Ответ:

- **Топик и объект заказа**
- Только ID пользователя
- Адрес Redis
- Только ключ сообщения

Шаг 03:

Вопрос 3 Расположите шаги отправки заказа из API-сервиса в Kafka:

1. Создать объект Producer
2. Настроить сериализацию и брокеры
3. Создать сообщение с данными заказа
4. Вызвать send
5. Закрыть Producer

Подсказка:

Сначала готовим Producer, затем формируем сообщение и отправляем его, после чего закрываем соединение.

Ответ:

- Создать объект Producer
- Настроить сериализацию и брокеры
- Создать сообщение с данными заказа

- Вызвать send
- Закрыть Producer

🔗 Шаг 04:

Вопрос 4

Что будет, если не закрыть Producer после отправки события?

Подсказка: Producer хранит буферы сообщений и соединения с брокером.

Ответ:

- **Некоторые сообщения могут не отправиться**
- Сообщения сохраняются автоматически в базе
- Producer закроется сам
- Сообщения будут дублироваться

🔗 Шаг 05:

Вопрос 5

Сопоставьте объекты Kafka с их ролью при отправке заказа:

Ответ:

- Producer
 - ProducerRecord
 - Serializer
1. Отправка сообщений в топик
 2. Формирование сообщения с ключом и значением
 3. Преобразование объекта заказа в байты

🔗 Подсказка:

Кто отвечает за отправку, кто формирует запись, кто сериализует.

- **Producer** — Отправка сообщений в топик
- **ProducerRecord** — Формирование сообщения с ключом и значением
- **Serializer** — Преобразование объекта заказа в байты

🔗 Шаг 06:

Вопрос 6

Какая настройка Producer гарантирует доставку события заказа?

Подсказка: Подумайте о подтверждениях от брокеров.

Ответ:

- **acks=all**
- enable.auto.commit=false
- group.id
- auto.offset.reset=latest

🔗 Шаг 07:

Вопрос 7 Расположите действия при ошибке отправки заказа:

1. Попытка отправки
2. Логирование ошибки

3. Повторная отправка или откат
4. Закрытие Producer

 Подсказка:

Сначала пробуем отправить, при ошибке логируем и предпринимаем действия для восстановления.

Ответ:

- Попытка отправки
- Логирование ошибки
- Повторная отправка или откат
- Закрытие Producer

 Шаг 08:

Вопрос 8

Какой параметр Producer важен для безопасной многопоточной отправки событий заказа?

Подсказка: Подумайте об идемпотентности.

Ответ:

- **enable.idempotence=true**
- `auto.offset.reset=earliest`
- `group.id`
- `retries=0`

 Шаг 09:

Вопрос 9

Сопоставьте тип ошибки при публикации заказа и действие разработчика:

Ответ:

- `NetworkException`
 - `SerializationException`
 - `TimeoutException`
1. Проверить соединение и брокеры
 2. Исправить сериализацию объекта заказа
 3. Настроить `retries` и увеличить таймаут

 Подсказка:

Разные ошибки требуют разных подходов: сеть, данные или таймаут.

- **NetworkException** — Проверить соединение и брокеры
- **SerializationException** — Исправить сериализацию объекта заказа
- **TimeoutException** — Настроить `retries` и увеличить таймаут

 Шаг 10:

Вопрос 10

Какой подход обеспечивает минимальную потерю данных при массовой отправке заказов?

Подсказка: Подумайте о комбинации идемпотентности и подтверждений.

Ответ:

- **enable.idempotence=true + acks=all**

- `auto.commit=false + retries=0`
- `group.id` уникальный
- `auto.offset.reset=latest`

Модуль 4 - Kafka Worker

Урок 1 - Подписка на Kafka-топики

Шаг 01:

Вопрос 1

Какой компонент отвечает за получение сообщений из Kafka-топика?

Подсказка: Он читает события, публикуемые Producer'ом.

Ответ:

- **Kafka Consumer**
- Kafka Producer
- API Controller
- Database Repository

Шаг 02:

Вопрос 2

Что обязательно нужно указать при подписке на Kafka-топик?

Подсказка: Иначе Consumer не будет получать события.

Ответ:

- **Название топика и `group.id`**
- Адрес Producer'a
- Пароль Redis
- ID события

Шаг 03:

Вопрос 3 Расположите шаги получения и обработки сообщений Consumer'ом:

1. Создание объекта Consumer
2. Подписка на топик
3. Пуллинг сообщений
4. Обработка сообщений
5. Закрытие Consumer

Подсказка:

Сначала создаём Consumer, подписываемся, получаем и обрабатываем сообщения, затем закрываем соединение.

Ответ:

- Создание объекта Consumer
- Подписка на топик
- Пуллинг сообщений
- Обработка сообщений
- Закрытие Consumer

🔗 Шаг 04:

Вопрос 4

Что произойдёт, если несколько Consumer'ов используют один group.id?

Подсказка: Сообщения будут распределяться между ними.

Ответ:

- **Сообщения разделятся между Consumer'ами**
- Каждый Consumer получит полную копию всех сообщений
- Consumer не сможет подписаться на топик
- Произойдёт дублирование событий в топике

🔗 Шаг 05:

Вопрос 5

Сопоставьте объекты Kafka Consumer с их ролью:

Ответ:

- Consumer
 - ConsumerRecord
 - Deserializer
1. Чтение сообщений из топика
 2. Представление отдельного сообщения с ключом и значением
 3. Преобразование байтов в объект заказа

🔗 **Подсказка:**

Разберитесь, кто читает, кто формирует запись и кто десериализует данные.

- **Consumer** — Чтение сообщений из топика
- **ConsumerRecord** — Представление отдельного сообщения с ключом и значением
- **Deserializer** — Преобразование байтов в объект заказа

🔗 Шаг 06:

Вопрос 6

Как избежать потери сообщений при сбое Consumer'а?

Подсказка: Подумайте о сохранении позиции в топике.

Ответ:

- **Включить auto.offset.commit или использовать ручное подтверждение**
- Пропускать все сообщения при сбое
- Повторная подписка на другой топик
- Использовать отдельный Producer для Consumer

🔗 Шаг 07:

Вопрос 7 Расположите шаги обработки сообщения Consumer'ом с ручным подтверждением:

1. Получение сообщения
2. Обработка заказа
3. Коммит offset
4. Обработка следующего сообщения

Подсказка:

Сначала получаем, потом обрабатываем и только после успешной обработки подтверждаем offset.

Ответ:

- Получение сообщения
- Обработка заказа
- Коммит offset
- Обработка следующего сообщения

Шаг 08:

Вопрос 8

Какой параметр Consumer нужен для чтения сообщений с начала топика?

Подсказка: Иногда нужно обработать все события, даже старые.

Ответ:

- **auto.offset.reset=earliest**
- enable.auto.commit=false
- group.id уникальный
- acks=all

Урок 2 - Масштабирование обработчиков (consumer groups)

Шаг 01:

Вопрос 1

Что позволяет нескольким Consumer'ам в одной группе обрабатывать сообщения параллельно?

Подсказка: Подумайте о разделении нагрузки.

Ответ:

- **Consumer group**
- Kafka Topic
- Partition key
- Producer client

Шаг 02:

Вопрос 2

Какое ограничение существует при распределении партиций между Consumer'ами в одной группе?

Подсказка: Один Consumer не может получать сообщения из двух партиций одновременно.

Ответ:

- **Количество Consumer не может превышать количество партиций**
- Количество партиций не может превышать 1
- Все Consumer получают все сообщения
- Consumer могут быть без group.id

Шаг 03:

Вопрос 3 Расположите шаги масштабирования Consumer'ов в группе:

1. Добавление нового Consumer в группу

2. Перераспределение партиций между Consumer'ами
3. Продолжение обработки сообщений без остановки системы

🔗 **Подсказка:**

Сначала добавляем, потом Kafka перераспределяет, затем обработка продолжается.

Ответ:

- Добавление нового Consumer в группу
- Перераспределение партиций между Consumer'ами
- Продолжение обработки сообщений без остановки системы

🔗 **Шаг 04:**

Вопрос 4

Что произойдёт, если Consumer внезапно отключится?

Подсказка: Подумайте о failover и перераспределении нагрузки.

Ответ:

- **Kafka перераспределит партиции оставшимся Consumer'ам**
- Сообщения потеряются без восстановления
- Consumer автоматически подключится к другому топик
- Группа Consumer будет удалена

🔗 **Шаг 05:**

Вопрос 5

Сопоставьте термин с правильным объяснением:

Ответ:

- Consumer group
 - Partition
 - Rebalance
1. Группа Consumer, работающих совместно
 2. Подраздел топика для параллельной обработки
 3. Перераспределение партиций при изменении группы

🔗 **Подсказка:**

Обратите внимание на роль каждого элемента при масштабировании.

- **Consumer group** — Группа Consumer, работающих совместно
- **Partition** — Подраздел топика для параллельной обработки
- **Rebalance** — Перераспределение партиций при изменении группы

🔗 **Шаг 06:**

Вопрос 6

Какой параметр гарантирует, что каждый Consumer получает только свои сообщения?

Подсказка: Смотрите на уникальность Consumer в группе.

Ответ:

- **group.id**
- client.id
- topic.name

- partition.id

Шаг 07:

Вопрос 7 Расположите действия при масштабировании группы с новыми Consumer'ами:

1. Добавление Consumer
2. Trigger Rebalance
3. Обработка сообщений новыми Consumer'ами
4. Мониторинг загрузки партиций

Подсказка:

Kafka сначала узнаёт о новых Consumer, перераспределяет партиции, затем начинается обработка и мониторинг.

Ответ:

- Добавление Consumer
- Trigger Rebalance
- Обработка сообщений новыми Consumer'ами
- Мониторинг загрузки партиций

Урок 3 - Валидация заказов

Шаг 01:

Вопрос 1

Что первым делом проверяет Kafka Worker при поступлении нового заказа?

Подсказка: Начинаем с базовой проверки данных.

Ответ:

- **Корректность данных заказа**
- Запуск доставки сразу
- Отправка уведомления клиенту
- Обновление статистики продаж

Шаг 02:

Вопрос 2

Какие поля заказа обычно проверяются на валидность в Kafka Worker?

Подсказка: Думайте о минимальном наборе данных для обработки.

Ответ:

- **ID товара**
- **Количество**
- **Адрес доставки**
- Почтовый индекс клиента соседнего города

Шаг 03:

Вопрос 3 Расположите шаги валидации заказа в правильном порядке в Kafka Worker:

1. Проверка данных заказа
2. Проверка наличия товара на складе
3. Проверка адреса доставки

4. Отправка заказа на обработку

Подсказка: Начинаем с базовой проверки, затем склад, адрес и очередь.

Ответ:

- Проверка данных заказа
- Проверка наличия товара на складе
- Проверка адреса доставки
- Отправка заказа на обработку

Шаг 04:

Вопрос 4

Что делает Kafka Worker, если заказ некорректен?

Подсказка: Подумайте о безопасной обработке ошибок.

Ответ:

- **Отправляет сообщение об ошибке и блокирует заказ**
- Игнорирует ошибку и обрабатывает заказ
- Удаляет все заказы клиента
- Автоматически исправляет ошибку

Шаг 05:

Вопрос 5

Сопоставьте тип ошибки заказа с описанием:

Ответ:

- Некорректный ID товара
 - Недостаточное количество
 - Неверный адрес доставки
 - Отсутствие оплаты
1. ID товара не найден
 2. Заказанное количество превышает наличие
 3. Адрес не соответствует формату
 4. Заказ не оплачен

 **Подсказка:**

Каждая ошибка связана с проверкой конкретного поля заказа.

- **Некорректный ID товара** — ID товара не найден
- **Недостаточное количество** — Заказанное количество превышает наличие
- **Неверный адрес доставки** — Адрес не соответствует формату
- **Отсутствие оплаты** — Заказ не оплачен

Шаг 06:

Вопрос 6

Почему важно валидировать заказ до отправки в Kafka топик?

Подсказка: Думайте о надежности системы и экономии ресурсов.

Ответ:

- **Чтобы избежать ошибок и лишней нагрузки на обработку**

- Чтобы ускорить оплату клиента
- Чтобы автоматически исправить ошибки заказа
- Чтобы уведомить всех клиентов одновременно

Шаг 07:

Вопрос 7 Расположите шаги обработки заказа после валидации:

1. Отправка заказа в Kafka топик
2. Проверка валидности данных
3. Обновление статуса заказа
4. Уведомление клиента

Подсказка: Сначала проверка, затем очередь, статус и уведомление.

Ответ:

- Проверка валидности данных
- Отправка заказа в Kafka топик
- Обновление статуса заказа
- Уведомление клиента

Урок 4 - Добавление бизнес-логики и расчетов

Шаг 01:

Вопрос 1

Что первым делом делает Kafka Worker при поступлении нового заказа для расчетов?

Подсказка: Начнем с базовой проверки данных заказа перед расчетами.

Ответ:

- **Проверяет корректность данных заказа**
- Сразу выполняет расчет скидок
- Отправляет уведомление клиенту
- Обновляет статистику продаж

Шаг 02:

Вопрос 2

Какие параметры заказа обычно проверяются перед расчетом бизнес-логики?

Подсказка: Подумайте о минимальном наборе для корректного расчета.

Ответ:

- **Цена товара**
- **Количество**
- **Скидка по промокоду**
- Цвет интерфейса приложения

Шаг 03:

Вопрос 3 Расположите шаги бизнес-логики при обработке заказа в Kafka Worker:

1. Проверка данных заказа
2. Расчет скидки и налогов
3. Обновление суммы к оплате

4. Отправка результата в топик Kafka

Подсказка: Начинаем с проверки, затем расчет, обновление и отправка.

Ответ:

- Проверка данных заказа
- Расчет скидки и налогов
- Обновление суммы к оплате
- Отправка результата в топик Kafka

Шаг 04:

Вопрос 4

Что делает Kafka Worker, если расчет бизнес-логики завершился с ошибкой?

Подсказка: Подумайте о безопасной обработке ошибок.

Ответ:

- **Отправляет сообщение об ошибке и блокирует заказ**
- Игнорирует ошибку и продолжает обработку
- Удаляет заказ из базы
- Автоматически исправляет данные заказа

Шаг 05:

Вопрос 5

Сопоставьте элемент бизнес-логики с его описанием:

Ответ:

- Проверка наличия товара
 - Расчет скидки
 - Вычисление налогов
 - Обновление суммы к оплате
1. Проверка доступности на складе
 2. Применение промокодов и акций
 3. Добавление НДС и других сборов
 4. Итоговая сумма заказа

 **Подсказка:**

Каждый элемент отвечает за отдельную часть расчета заказа.

- **Проверка наличия товара** — Проверка доступности на складе
- **Расчет скидки** — Применение промокодов и акций
- **Вычисление налогов** — Добавление НДС и других сборов
- **Обновление суммы к оплате** — Итоговая сумма заказа

Шаг 06:

Вопрос 6

Почему важно валидировать входные данные перед расчетами бизнес-логики?

Подсказка: Думайте о надежности и точности результатов.

Ответ:

- **Чтобы избежать некорректных расчетов и ошибок в топике Kafka**

- Чтобы ускорить отправку уведомлений клиенту
- Чтобы автоматически исправить все ошибки
- Чтобы изменить формат заказа

🔗 Шаг 07:

Вопрос 7 Расположите шаги обработки бизнес-логики после валидации:

1. Расчет скидок и налогов
2. Проверка валидности заказа
3. Обновление суммы заказа
4. Отправка заказа в Kafka топик

Подсказка: Сначала проверка, затем расчет, обновление и отправка.

Ответ:

- Проверка валидности заказа
- Расчет скидок и налогов
- Обновление суммы заказа
- Отправка заказа в Kafka топик

🔗 Шаг 08:

Вопрос 8

Что происходит, если расчет налогов в Kafka Worker завершился с ошибкой?

Подсказка: Подумайте о корректной обработке ошибок и логировании.

Ответ:

- **Отправляется ошибка и заказ блокируется**
- Ошибка игнорируется и заказ идет дальше
- Заказ удаляется из топика
- Происходит автоматическая коррекция налогов

🔗 Шаг 09:

Вопрос 9

Сопоставьте элементы бизнес-логики с соответствующей последовательностью:

Ответ:

- Проверка заказа
- Расчет скидки
- Вычисление налогов
- Отправка результата

1. 1
2. 2
3. 3
4. 4

🔗 **Подсказка:**

Подумайте о логическом порядке шагов.

- **Проверка заказа — 1**
- **Расчет скидки — 2**

- **Вычисление налогов** — 3
- **Отправка результата** — 4

Урок 5 - Практика разработка Worker-сервиса, обрабатывающего заказы.

Шаг 01:

Вопрос 1

Что является основной задачей Worker-сервиса в event-driven пайплайне?

Подсказка: Подумайте, что делает сервис с приходящими сообщениями.

Ответ:

- **Обрабатывает сообщения из Kafka и выполняет бизнес-логику**
- Отправляет сообщения в API-сервис
- Создает топики в Kafka
- Мониторит базу данных без обработки сообщений

Шаг 02:

Вопрос 2

Как Worker узнаёт, что пришло новое сообщение для обработки?

Подсказка: Смотрите на подписку на топики.

Ответ:

- **Через подписку на Kafka-топик**
- Через чтение локального файла
- Через HTTP polling API-сервиса
- Через планировщик cron

Шаг 03:

Вопрос 3 Расположите шаги обработки заказа Worker-сервисом:

1. Получение сообщения из топика
2. Выполнение бизнес-логики (например, обработка заказа)
3. Подтверждение обработки и commit offset

Подсказка:

Сначала Worker получает сообщение, затем обрабатывает и только потом фиксирует прогресс.

Ответ:

- Получение сообщения из топика
- Выполнение бизнес-логики (например, обработка заказа)
- Подтверждение обработки и commit offset

Шаг 04:

Вопрос 4

Что произойдёт, если Worker не выполнит commit offset после обработки сообщения?

Подсказка: Подумайте, как Kafka отслеживает прогресс обработки сообщений.

Ответ:

- **При следующем запуске Worker может повторно обработать это сообщение**

- Сообщение исчезнет навсегда
- Kafka создаст новый топик
- Worker автоматически выполнит commit offset

🔮 Шаг 05:

Вопрос 5

Сопоставьте элементы Worker-сервиса с их ролью:

Ответ:

- Kafka Consumer
 - Business Logic
 - Offset Commit
1. Получает сообщения из топика
 2. Обрабатывает данные и выполняет действия
 3. Фиксирует прогресс обработки сообщений

🔮 Подсказка:

Каждый элемент имеет свою отдельную ответственность в обработке сообщений.

- **Kafka Consumer** — Получает сообщения из топика
- **Business Logic** — Обрабатывает данные и выполняет действия
- **Offset Commit** — Фиксирует прогресс обработки сообщений

🔮 Шаг 06:

Вопрос 6

Что важно учитывать при параллельной работе нескольких Worker'ов на один топик?

Подсказка: Подумайте о распределении партиций.

Ответ:

- **Каждый Worker должен обрабатывать только свои партиции**
- Все Worker обрабатывают все сообщения параллельно
- Worker не могут работать параллельно на одном топике
- Worker автоматически создают новые топики при загрузке

🔮 Шаг 07:

Вопрос 7 Расположите события при сбое одного Worker'a:

1. Worker отключается
2. Kafka обнаруживает отсутствие heartbeat
3. Партиции перераспределяются между оставшимися Worker'ами
4. Сообщения продолжают обрабатываться

🔮 Подсказка:

Сначала Worker падает, потом Kafka реагирует и перераспределяет нагрузку.

Ответ:

- Worker отключается
- Kafka обнаруживает отсутствие heartbeat
- Партиции перераспределяются между оставшимися Worker'ами
- Сообщения продолжают обрабатываться

Модуль 5 - Кэширование с Redis

Урок 1 - Зачем нужен кэш при потоковой обработке

Шаг 01:

Вопрос 1

Почему при потоковой обработке заказов удобно использовать кэш?

Подсказка: Подумайте о скорости доступа к часто используемым данным.

Ответ:

- **Чтобы ускорить доступ к часто используемым данным**
- Чтобы хранить данные навсегда
- Чтобы полностью заменить базу данных
- Чтобы отправлять уведомления клиентам

Шаг 02:

Вопрос 2

Какие данные обычно кэшируют при потоковой обработке?

Подсказка: Подумайте о повторяющихся вычислениях и часто запрашиваемых значениях.

Ответ:

- **Часто запрашиваемые результаты вычислений**
- **Статусы заказов, используемые несколько раз**
- Секретные пароли пользователей
- Логи всех запросов на диске

Шаг 03:

Вопрос 3 Расположите шаги работы Worker с кэшем Redis:

1. Проверка, есть ли данные в кэше
2. Если есть, взять их из кэша
3. Если нет, получить данные из базы
4. Сохранить данные в кэше
5. Продолжить обработку заказа

Подсказка: Сначала проверка, затем получение, сохранение и обработка.

Ответ:

- Проверка, есть ли данные в кэше
- Если есть, взять их из кэша
- Если нет, получить данные из базы
- Сохранить данные в кэше
- Продолжить обработку заказа

Шаг 04:

Вопрос 4

Что произойдет, если кэш Redis не используется?

Подсказка: Подумайте о нагрузке на базу и скорости обработки.

Ответ:

- **Каждое обращение идет к базе, что снижает производительность**
- Данные становятся автоматически постоянными
- Worker не сможет обрабатывать заказы
- Redis сам создает кэш без кода

🔗 **Шаг 05:**

Вопрос 5

Сопоставьте элементы кэширования с их назначением:

Ответ:

- Redis
 - Ключ
 - Значение
 - TTL (время жизни)
1. Система хранения быстрых данных
 2. Идентификатор данных в кэше
 3. Содержимое данных
 4. Время, через которое данные удаляются

🔗 **Подсказка:**

Подумайте о структуре хранения данных в Redis.

- **Redis** — Система хранения быстрых данных
- **Ключ** — Идентификатор данных в кэше
- **Значение** — Содержимое данных
- **TTL (время жизни)** — Время, через которое данные удаляются

🔗 **Шаг 06:**

Вопрос 6

Почему важно задавать TTL для данных в Redis?

Подсказка: Подумайте о памяти и актуальности данных.

Ответ:

- **Чтобы не перегружать память и держать данные актуальными**
- Чтобы Redis мог обрабатывать заказы быстрее
- Чтобы автоматически обновлять базу данных
- Чтобы Worker мог пропускать шаги обработки

🔗 **Шаг 07:**

Вопрос 7 Расположите шаги кеширования данных в Redis:

1. Проверка наличия данных в кэше
2. **Если данных нет** — извлечение из базы
3. Сохранение данных с TTL
4. Использование данных для обработки

Подсказка: Сначала проверка, затем извлечение, сохранение и обработка.

Ответ:

- Проверка наличия данных в кэше
- **Если данных нет** — извлечение из базы
- Сохранение данных с TTL
- Использование данных для обработки

Шаг 08:

Вопрос 9

Сопоставьте элементы Worker с последовательностью обработки заказа:

Ответ:

- Валидация заказа
- Расчет стоимости
- Обновление заказа
- Отправка результата

1. 1
2. 2
3. 3
4. 4

Подсказка:

Подумайте о логическом порядке шагов.

- **Валидация заказа** — 1
- **Расчет стоимости** — 2
- **Обновление заказа** — 3
- **Отправка результата** — 4

Шаг 09:

Вопрос 10

Какой принцип гарантирует корректную обработку заказов Worker-сервисом?

Подсказка: Думайте о надежности, последовательности и точности вычислений.

Ответ:

- **Последовательное выполнение всех шагов обработки с проверкой на каждом этапе**
- Пропуск промежуточных проверок для ускорения
- Автоматическая коррекция всех ошибок без логирования
- Случайный порядок выполнения шагов

Урок 2 - Redis как in-memory база данных

Шаг 01:

Вопрос 1

Почему Redis называют in-memory базой данных?

Подсказка: Подумайте о том, где хранятся данные для быстрого доступа.

Ответ:

- **Данные хранятся в оперативной памяти для высокой скорости доступа**

- Данные хранятся только на диске
- Redis использует только облачное хранилище
- Redis хранит данные в базе SQL

🔮 Шаг 02:

Вопрос 2

Какие типы данных поддерживает Redis?

Подсказка: Подумайте о ключ-значение и сложных структурах данных.

Ответ:

- Строки (String)
- Списки (List)
- Хэши (Hash)
- Множества (Set)
- Таблицы SQL

🔮 Шаг 03:

Вопрос 3 Расположите шаги записи данных в Redis:

1. Создать ключ
2. Определить значение
3. Сохранить ключ-значение в Redis
4. Задать TTL (если нужно)

Подсказка: Сначала ключ, затем значение, сохранение и опциональный TTL.

Ответ:

- Создать ключ
- Определить значение
- Сохранить ключ-значение в Redis
- Задать TTL (если нужно)

🔮 Шаг 04:

Вопрос 4

Что происходит, если ключ в Redis с TTL истекает?

Подсказка: Подумайте о доступности устаревших данных.

Ответ:

- Ключ автоматически удаляется и данные становятся недоступными
- Данные сохраняются навсегда
- Redis генерирует ошибку при доступе к ключу
- TTL обновляется автоматически до следующего обращения

🔮 Шаг 05:

Вопрос 5

Сопоставьте команды Redis с их действием:

Ответ:

- SET
- GET

- DEL
- EXPIRE

1. Сохранить ключ-значение
2. Получить значение по ключу
3. Удалить ключ
4. Задать TTL для ключа

🔗 Подсказка:

Подумайте, что делают стандартные команды Redis для работы с ключами.

- **SET** — Сохранить ключ-значение
- **GET** — Получить значение по ключу
- **DEL** — Удалить ключ
- **EXPIRE** — Задать TTL для ключа

🔗 Шаг 06:

Вопрос 6

Почему Redis быстрее традиционной базы данных?

Подсказка: Подумайте о хранении данных и чтении/записи.

Ответ:

- **Все данные хранятся в памяти и операции выполняются напрямую без дисковых запросов**
- Redis использует сложные индексы на диске
- Redis хранит данные на SSD и выполняет кэширование автоматически
- Redis выполняет SQL-запросы быстрее других СУБД

🔗 Шаг 07:

Вопрос 7 Расположите шаги работы Worker с Redis при обработке заказов:

1. Проверка кэша Redis
2. **Если данных нет** — получить их из базы
3. Сохранение в Redis с TTL
4. Использование данных для обработки заказа

Подсказка: Сначала проверяем кэш, затем извлекаем, сохраняем и обрабатываем.

Ответ:

- Проверка кэша Redis
- **Если данных нет** — получить их из базы
- Сохранение в Redis с TTL
- Использование данных для обработки заказа

🔗 Шаг 08:

Вопрос 8

Что произойдет, если несколько воркеров одновременно сохраняют один и тот же ключ в Redis?

Подсказка: Подумайте о свойствах Redis при записи данных.

Ответ:

- **Последняя запись перезапишет предыдущую**

- Произойдет ошибка и данные потеряются
- Redis создаст несколько копий ключа
- Старые данные автоматически сохранятся в резерве

🔔 Шаг 09:

Вопрос 9

Сопоставьте преимущества Redis с их объяснением:

Ответ:

- Скорость
 - Поддержка структур данных
 - TTL
 - Atomic операции
1. Очень быстрый доступ к данным в памяти
 2. Возможность хранить строки, списки, хэши, множества
 3. Автоматическое удаление устаревших данных
 4. Безопасные операции без конфликтов при параллельной работе

🔔 Подсказка:

Подумайте, почему Redis так удобен для потоковой обработки.

- **Скорость** — Очень быстрый доступ к данным в памяти
- **Поддержка структур данных** — Возможность хранить строки, списки, хэши, множества
- **TTL** — Автоматическое удаление устаревших данных
- **Atomic операции** — Безопасные операции без конфликтов при параллельной работе

📖 Урок 3 - Подключение Redis в микросервис

🔔 Шаг 01:

Вопрос 1

Какой протокол используется для подключения к Redis?

Подсказка: Подумайте о стандартном клиент-серверном взаимодействии Redis.

Ответ:

- **RESP (Redis Serialization Protocol)**
- HTTP
- FTP
- SMTP

🔔 Шаг 02:

Вопрос 2

Что необходимо для подключения микросервиса к Redis?

Подсказка: Подумайте о стандартной конфигурации клиента.

Ответ:

- **Адрес сервера Redis**
- **Порт сервера**
- **Ключ аутентификации (если установлен)**
- SQL-запрос для подключения

- SMTP настройки

Шаг 03:

Вопрос 3 Расположите шаги подключения микросервиса к Redis:

1. Импорт библиотеки Redis-клиента
2. Создание экземпляра клиента с параметрами подключения
3. Установка соединения с сервером
4. Использование клиента для операций с данными

Подсказка: Сначала импорт, затем конфигурация, соединение и работа с данными.

Ответ:

- Импорт библиотеки Redis-клиента
- Создание экземпляра клиента с параметрами подключения
- Установка соединения с сервером
- Использование клиента для операций с данными

Шаг 04:

Вопрос 4

Что произойдет, если клиент не сможет подключиться к Redis?

Подсказка: Подумайте о поведении микросервиса при отсутствии соединения.

Ответ:

- **Клиент выдаст ошибку подключения, и операции с кэшем будут недоступны**
- Redis автоматически создаст новое соединение без ошибок
- Микросервис продолжит работу без использования данных
- Данные будут сохраняться локально на клиенте

Шаг 05:

Вопрос 5

Сопоставьте параметры Redis-клиента с их назначением:

Ответ:

- host
 - port
 - password
 - db
1. Адрес сервера Redis
 2. Порт подключения
 3. Пароль для аутентификации
 4. Выбор конкретной базы Redis

Подсказка:

Подумайте, какие параметры обычно указываются при создании клиента Redis.

- **host** — Адрес сервера Redis
- **port** — Порт подключения
- **password** — Пароль для аутентификации
- **db** — Выбор конкретной базы Redis

🔗 Шаг 06:

Вопрос 6

Почему важно правильно настраивать пул соединений Redis для микросервиса?

Подсказка: Подумайте о нагрузке и параллельных запросах.

Ответ:

- **Чтобы избежать исчерпания соединений и задержек при высокой нагрузке**
- Чтобы Redis автоматически создавал новые базы данных
- Чтобы микросервис мог работать без клиента Redis
- Чтобы данные сохранялись навсегда без TTL

🔗 Шаг 07:

Вопрос 7 Расположите этапы обработки запроса микросервиса с Redis:

1. Проверка кэша Redis
2. **Если данных нет** — получить их из основной базы
3. Сохранение данных в Redis с TTL
4. Отправка ответа клиенту

Подсказка: Сначала ищем в кэше, потом при необходимости извлекаем из базы, кэшируем и отвечаем.

Ответ:

- Проверка кэша Redis
- **Если данных нет** — получить их из основной базы
- Сохранение данных в Redis с TTL
- Отправка ответа клиенту

🔗 Шаг 08:

Вопрос 8

Какие проблемы могут возникнуть при неправильном подключении к Redis?

Подсказка: Подумайте о доступности данных и производительности микросервиса.

Ответ:

- **Ошибка подключения и недоступность кэша**
- **Высокие задержки при обработке запросов**
- Redis создаст резервную копию автоматически
- Все данные сохранятся в локальной памяти без проблем

🔗 Шаг 09:

Вопрос 9

Сопоставьте действия микросервиса с использованием Redis с их эффектом:

Ответ:

- GET ключа
- SET ключа с TTL
- DEL ключа
- INCR ключа

1. Получение данных из кэша

2. Сохранение данных в кэш на ограниченное время
3. Удаление устаревшего ключа
4. Увеличение числового значения ключа

 Подсказка:

Подумайте о стандартных командах Redis и их эффекте на данные.

- **GET ключа** — Получение данных из кэша
- **SET ключа с TTL** — Сохранение данных в кэш на ограниченное время
- **DEL ключа** — Удаление устаревшего ключа
- **INCR ключа** — Увеличение числового значения ключа

 Шаг 10:

Вопрос 10

Как Redis помогает микросервису выдерживать высокую нагрузку?

Подсказка: Подумайте о кэшировании повторяющихся запросов и уменьшении обращений к базе данных.

Ответ:

- **Хранение результатов частых запросов в памяти**
- **Снижение числа обращений к основной базе данных**
- Redis увеличивает производительность процессора автоматически
- Redis заменяет SQL-сервер полностью

 Урок 4 - Сохранение результата обработки заказа

 Шаг 01:

Вопрос 1

Какая основная причина использовать Redis для сохранения результатов обработки заказа?

Подсказка: Подумайте о скорости доступа к данным.

Ответ:

- **Быстрый доступ к часто используемым данным**
- Хранение больших файлов
- Автоматическое создание SQL-запросов
- Обработка email уведомлений

 Шаг 02:

Вопрос 2

Что необходимо указать при сохранении результата заказа в Redis?

Подсказка: Подумайте о ключах и значениях.

Ответ:

- **Уникальный ключ для заказа**
- **Данные заказа в формате JSON**
- **Время жизни (TTL) для кэширования**
- SQL-запрос для записи
- SMTP настройки для уведомлений

🔗 Шаг 03:

Вопрос 3 Расположите шаги сохранения результата обработки заказа в Redis:

1. Формирование уникального ключа заказа
2. Сериализация данных заказа в JSON
3. Сохранение данных в Redis с TTL
4. Проверка успешного сохранения

Подсказка: Сначала формируем ключ, затем сериализуем, сохраняем и проверяем.

Ответ:

- Формирование уникального ключа заказа
- Сериализация данных заказа в JSON
- Сохранение данных в Redis с TTL
- Проверка успешного сохранения

🔗 Шаг 04:

Вопрос 4

Что произойдет, если TTL для результата заказа не установлен?

Подсказка: Подумайте о кэше и использовании памяти.

Ответ:

- **Данные останутся в памяти Redis до ручного удаления, что может вызвать переполнение**
- Redis автоматически очистит данные через 5 минут
- Данные будут храниться только на диске
- Микросервис перестанет работать

🔗 Шаг 05:

Вопрос 5

Сопоставьте команды Redis с их назначением при работе с результатами заказов:

Ответ:

- SET
 - GET
 - EXPIRE
 - DEL
1. Сохранение данных
 2. Получение данных
 3. Установка времени жизни ключа
 4. Удаление ключа

🔗 **Подсказка:**

Подумайте, какие команды используются для управления ключами и значениями.

- **SET** — Сохранение данных
- **GET** — Получение данных
- **EXPIRE** — Установка времени жизни ключа
- **DEL** — Удаление ключа

🔗 Шаг 06:

Вопрос 6

Почему важно проверять успешное сохранение результата заказа в Redis?

Подсказка: Подумайте о консистентности данных и опыте пользователя.

Ответ:

- Чтобы убедиться, что клиент получит корректный результат при следующем запросе
- Чтобы Redis автоматически создавал резервную копию
- Чтобы данные хранились на диске
- Чтобы избежать ошибок SQL-запросов

🔗 Шаг 07:

Вопрос 7 Расположите шаги обработки повторного запроса заказа с кэшированием:

1. Проверка кэша Redis на наличие результата
2. **Если результат найден** — возвращение клиенту
3. **Если результата нет** — обработка заказа и сохранение в Redis
4. Отправка ответа клиенту

Подсказка: Сначала проверяем кэш, потом при необходимости обрабатываем и сохраняем.

Ответ:

- Проверка кэша Redis на наличие результата
- **Если результат найден** — возвращение клиенту
- **Если результата нет** — обработка заказа и сохранение в Redis
- Отправка ответа клиенту

🔗 Шаг 08:

Вопрос 8

Что может произойти при неправильной сериализации данных заказа в Redis?

Подсказка: Подумайте о возможности восстановления информации при чтении из кэша.

Ответ:

- Данные не будут корректно десериализованы при чтении
- Может произойти потеря информации
- Redis исправит сериализацию автоматически
- Данные будут храниться в SQL автоматически

🔗 Шаг 09:

Вопрос 9

Сопоставьте типичные ошибки при работе с результатами заказов в Redis с их последствиями:

Ответ:

- Отсутствие TTL
 - Неправильная сериализация
 - Повторное использование ключа
 - Не проверка успешного сохранения
1. Переполнение памяти и устаревшие данные

2. Невозможность восстановления данных
3. Перезапись существующих результатов
4. Клиент получает некорректный результат

🔗 Подсказка:

Подумайте о последствиях каждой ошибки для кэша и данных заказов.

- **Отсутствие TTL** — Переполнение памяти и устаревшие данные
- **Неправильная сериализация** — Невозможность восстановления данных
- **Повторное использование ключа** — Перезапись существующих результатов
- **Не проверка успешного сохранения** — Клиент получает некорректный результат

🔗 Шаг 10:

Вопрос 10

Как использование Redis улучшает опыт пользователя при повторных заказах?

Подсказка: Подумайте о скорости получения результатов и нагрузке на базу.

Ответ:

- **Результаты заказа возвращаются мгновенно из кэша**
- **Снижается нагрузка на основную базу данных**
- Redis заменяет микросервис полностью
- Клиент может редактировать кэш напрямую

📖 Урок 5 - Практика Worker сохраняет обработанный заказ в Redis.

🔗 Шаг 01:

Вопрос 1

Какова основная цель использования Redis в Worker-сервисе?

Подсказка: Подумайте о скорости доступа к данным.

Ответ:

- **Быстрое хранение и получение обработанных заказов**
- Организация очереди сообщений
- Мониторинг Worker-сервиса
- Обработка бизнес-логики

🔗 Шаг 02:

Вопрос 2

Как Worker записывает результат обработки заказа в Redis?

Подсказка: Подумайте о структуре данных и ключах.

Ответ:

- **Сохраняет данные по уникальному ключу заказа**
- Использует HTTP-запрос к Redis
- Записывает данные в файл и Redis синхронизирует их
- Сохраняет данные в базу Kafka

🔗 Шаг 03:

Вопрос 3 Расположите шаги записи заказа в Redis:

1. Worker обрабатывает заказ
2. Формируется уникальный ключ для Redis
3. Данные записываются в Redis

🔗 Подсказка:

Сначала обработка, затем ключ, потом запись в Redis.

Ответ:

- Worker обрабатывает заказ
- Формируется уникальный ключ для Redis
- Данные записываются в Redis

🔗 Шаг 04:

Вопрос 4

Что произойдет, если Worker попытается записать заказ в Redis, а ключ уже существует?

Подсказка: Подумайте о поведении Redis при записи с существующим ключом.

Ответ:

- **Старое значение перезаписывается новым**
- Запись игнорируется автоматически
- Redis создаёт новый ключ с постфиксом
- Worker упадет с ошибкой

🔗 Шаг 05:

Вопрос 5

Сопоставьте команды Redis и их роль:

Ответ:

- SET
 - GET
 - DEL
1. Сохраняет значение по ключу
 2. Возвращает значение по ключу
 3. Удаляет ключ

🔗 Подсказка:

Каждая команда Redis имеет свою простую и понятную функцию.

- **SET** — Сохраняет значение по ключу
- **GET** — Возвращает значение по ключу
- **DEL** — Удаляет ключ

🔗 Шаг 06:

Вопрос 6

Как обеспечить, чтобы Worker не потерял данные при сбое Redis?

Подсказка: Подумайте о настройках persistence.

Ответ:

- **Включить snapshotting или AOF для Redis**
- Полагаться только на кеш в памяти Worker'a
- Писать данные только в Kafka
- Не использовать Redis вовсе

🔗 **Шаг 07:**

Вопрос 7 Расположите действия при чтении обработанного заказа из Redis:

1. Worker формирует ключ заказа
2. Worker выполняет GET по ключу
3. Полученное значение используется для дальнейшей обработки

🔗 **Подсказка:**

Сначала формируем ключ, затем читаем значение, потом используем его.

Ответ:

- Worker формирует ключ заказа
- Worker выполняет GET по ключу
- Полученное значение используется для дальнейшей обработки

🔗 **Шаг 08:**

Вопрос 8

Что помогает Redis хранить данные с ограничением по времени (TTL)?

Подсказка: Подумайте о автоматическом удалении устаревших записей.

Ответ:

- **Использование команды EXPIRE или SET с параметром PX/EX**
- Удаление вручную каждый раз
- Настройка Kafka retention
- Worker сам отслеживает устаревшие ключи

🔗 **Шаг 09:**

Вопрос 9

Сопоставьте тип данных Redis с примером использования:

Ответ:

- String
 - Hash
 - List
1. Хранение отдельного значения заказа
 2. Сохранение полей заказа (id, статус, сумма)
 3. Список обработанных заказов в очереди

🔗 **Подсказка:**

Каждый тип данных подходит под свой формат хранения.

- **String** — Хранение отдельного значения заказа
- **Hash** — Сохранение полей заказа (id, статус, сумма)
- **List** — Список обработанных заказов в очереди

Шаг 10:

Вопрос 10

Какой подход гарантирует корректную запись и чтение заказов при масштабировании Worker'ов с Redis?

Подсказка: Подумайте о согласованности и уникальных ключах.

Ответ:

- **Использовать уникальные ключи для каждого заказа и атомарные операции Redis**
- Все Worker пишут в один ключ
- Не использовать Redis при масштабировании
- Worker не проверяет, существует ли ключ

Модуль 6 - Client быстрый доступ к данным

Урок 1 - Реализация клиента для получения готовых данных

Шаг 01:

Вопрос 1

Что из ниже перечисленного является основным назначением gRPC клиента?

Подсказка: Клиент получает данные с сервера через удалённые вызовы.

Ответ:

- **Выполнение удалённых вызовов к серверу для получения данных**
- Запуск Kafka брокера
- Хранение данных в Redis
- Создание Docker контейнеров

Шаг 02:

Вопрос 2

Какие преимущества использования gRPC клиента для получения данных?

Подсказка: Подумайте о скорости, структуре и совместимости.

Ответ:

- **Высокая скорость передачи данных**
- **Строгая структура сообщений**
- **Поддержка нескольких языков программирования**
- Автоматическое создание Kafka топиков

Шаг 03:

Вопрос 3 : Расположите шаги получения данных через gRPC клиента в правильном порядке:

1. Создание gRPC соединения с сервером
2. Формирование запроса с нужными параметрами
3. Отправка запроса и получение ответа
4. Обработка и использование полученных данных

 **Подсказка:**

Сначала нужно соединиться, затем подготовить запрос, отправить его и обработать результат.

Ответ:

- Создание gRPC соединения с сервером
- Формирование запроса с нужными параметрами
- Отправка запроса и получение ответа
- Обработка и использование полученных данных

 Шаг 04:

 Вопрос 4

Сопоставьте понятия клиента gRPC с их назначением:

Ответ:

- Stub
 - Channel
 - Proto-файл
 - Unary вызов
1. Удалённая функция, доступная через клиент
 2. Сетевое соединение с сервером
 3. Описание структуры сообщений и сервисов
 4. **Один запрос** — один ответ

 Подсказка:

Подумайте, как клиент знает, какие методы доступны, как соединяется с сервером и как отправляет данные.

- **Stub** — Удалённая функция, доступная через клиент
- **Channel** — Сетевое соединение с сервером
- **Proto-файл** — Описание структуры сообщений и сервисов
- **Unary вызов** — Один запрос — один ответ

 Шаг 05:

Вопрос 5

При использовании клиента с Redis для кэширования данных, что важно учесть?

Подсказка: Думайте о свежести данных и производительности.

Ответ:

- **Не хранить устаревшие данные дольше TTL**
- Игнорировать ограничения памяти Redis
- Кэшировать все данные без фильтров
- Выключать клиент при пике нагрузки

 Шаг 06:

Вопрос 6 : Расположите шаги кэширования данных через Redis в правильном порядке:

1. Проверка наличия данных в кэше
2. Если данных нет, запрос к gRPC серверу
3. Сохранение полученных данных в Redis с TTL
4. Возврат данных клиенту

 Подсказка:

Сначала проверяем кэш, если данных нет — получаем и сохраняем, затем отдаём клиенту.

Ответ:

- Проверка наличия данных в кэше
- Если данных нет, запрос к gRPC серверу
- Сохранение полученных данных в Redis с TTL
- Возврат данных клиенту

Шаг 07:

Вопрос 7

Какие типы взаимодействия поддерживает gRPC клиент?

Подсказка: Подумайте о синхронных и потоковых способах.

Ответ:

- **Unary (один запрос — один ответ)**
- **Server streaming (сервер шлёт несколько сообщений)**
- **Client streaming (клиент шлёт несколько сообщений)**
- **Bidirectional streaming (двухнаправленный поток)**

Шаг 08:

Вопрос 8

Сопоставьте типы вызовов gRPC с их особенностями:

Ответ:

- Unary
 - Server streaming
 - Client streaming
 - Bidirectional streaming
1. **Один запрос** — один ответ
 2. Сервер отправляет несколько сообщений клиенту
 3. Клиент отправляет несколько сообщений серверу
 4. Клиент и сервер обмениваются сообщениями одновременно

 **Подсказка:**

Подумайте, кто инициирует поток и сколько сообщений отправляется.

- **Unary** — Один запрос — один ответ
- **Server streaming** — Сервер отправляет несколько сообщений клиенту
- **Client streaming** — Клиент отправляет несколько сообщений серверу
- **Bidirectional streaming** — Клиент и сервер обмениваются сообщениями одновременно

Шаг 09:

Вопрос 9

При работе с клиентом gRPC и Redis нужно учитывать:

Подсказка: Подумайте о производительности, надёжности и консистентности данных.

Ответ:

- **Обновление кэша при изменении данных на сервере**
- **Обработка ошибок соединения с сервером**

- **Использование TTL для данных в Redis**
- Игнорирование задержек в сети

Шаг 10:

Вопрос 10 : Расположите шаги работы клиентского пайплайна с gRPC, Redis и обработкой данных в правильном порядке:

1. Клиент проверяет данные в Redis
2. Если данных нет, клиент отправляет запрос к gRPC серверу
3. gRPC сервер возвращает данные клиенту
4. Клиент сохраняет данные в Redis с TTL
5. Клиент использует данные для обработки и отображения

Подсказка:

Сначала проверяем кэш, потом получаем с сервера при необходимости, сохраняем и используем данные.

Ответ:

- Клиент проверяет данные в Redis
- Если данных нет, клиент отправляет запрос к gRPC серверу
- gRPC сервер возвращает данные клиенту
- Клиент сохраняет данные в Redis с TTL
- Клиент использует данные для обработки и отображения

Урок 2 - Чтение из Redis

Шаг 01:

Вопрос 1

Какую роль выполняет Redis в клиентском пайплайне?

Подсказка: Думайте о скорости доступа к данным.

Ответ:

- **Кэширование часто используемых данных для быстрого доступа**
- Сохранение gRPC сообщений в очереди
- Аналитическая обработка больших данных
- Организация потоковой передачи данных в Kafka

Шаг 02:

Вопрос 2

Что важно учитывать при чтении данных из Redis клиентом?

Подсказка: Подумайте о свежести данных и ограничениях памяти.

Ответ:

- **Использовать TTL для актуальности данных**
- **Обрабатывать ошибки соединения с Redis**
- Игнорировать лимиты памяти Redis
- **Проверять наличие данных перед запросом к серверу**

Шаг 03:

Вопрос 3: Расположите шаги чтения данных из Redis клиентом в правильном порядке:

1. Проверка кэша на наличие нужных данных
2. Если данных нет, запрос к gRPC серверу
3. Обработка полученных данных
4. Сохранение данных в Redis с TTL

Подсказка:

Сначала проверяем кэш, потом при необходимости получаем с сервера, сохраняем и используем данные.

Ответ:

- Проверка кэша на наличие нужных данных
- Если данных нет, запрос к gRPC серверу
- Обработка полученных данных
- Сохранение данных в Redis с TTL

Шаг 04:

Вопрос 4

Сопоставьте команды Redis с их назначением:

Ответ:

- GET
 - SET
 - DEL
 - EXPIRE
1. Получить значение по ключу
 2. Установить значение по ключу
 3. Удалить ключ и значение
 4. Установить время жизни ключа

Подсказка:

Подумайте, как клиент получает, сохраняет и управляет временем жизни данных.

- **GET** — Получить значение по ключу
- **SET** — Установить значение по ключу
- **DEL** — Удалить ключ и значение
- **EXPIRE** — Установить время жизни ключа

Шаг 05:

Вопрос 5

Как клиент gRPC использует Redis для ускорения доступа к данным?

Подсказка: Подумайте о повторных запросах и кэше.

Ответ:

- **Сохраняет ранее полученные данные и проверяет кэш перед новым запросом**
- Игнорирует кэш и всегда обращается к серверу
- Использует Redis только для логирования
- Удаляет данные сразу после чтения

Шаг 06:

Вопрос 6: Расположите шаги обновления данных в Redis после изменения на сервере:

1. Сервер обновляет данные
2. Клиент получает обновленные данные через gRPC
3. Клиент сохраняет новые данные в Redis с TTL
4. Клиент использует свежие данные

Подсказка:

Сначала сервер меняет данные, клиент получает их и обновляет кэш.

Ответ:

- Сервер обновляет данные
- Клиент получает обновленные данные через gRPC
- Клиент сохраняет новые данные в Redis с TTL
- Клиент использует свежие данные

Шаг 07:

Вопрос 7

Какие риски возникают при чтении данных напрямую с Redis без проверки TTL?

Подсказка: Подумайте о достоверности данных.

Ответ:

- **Можно получить устаревшие данные**
- **Нарушается согласованность с сервером**
- Redis может потерять соединение с Kafka
- Данные автоматически обновляются без клиента

Шаг 08:

Вопрос 8

Сопоставьте ситуации с правильным действием при чтении данных из Redis:

Ответ:

- Данные есть в кэше
 - Данных нет в кэше
 - Данные устарели
 - Redis недоступен
1. Использовать данные напрямую
 2. Запросить данные у gRPC сервера и сохранить в Redis
 3. Обновить данные с сервера и перезаписать в кэше
 4. Обращать ошибку и, возможно, обратиться к резервному источнику

Подсказка:

Подумайте о кэш-хит/кэш-мисс и обработке ошибок.

- **Данные есть в кэше** — Использовать данные напрямую
- **Данных нет в кэше** — Запросить данные у gRPC сервера и сохранить в Redis
- **Данные устарели** — Обновить данные с сервера и перезаписать в кэше
- **Redis недоступен** — Обращать ошибку и, возможно, обратиться к резервному источнику

🔗 Шаг 09:

Вопрос 9

Что помогает оптимизировать чтение данных из Redis в клиентском пайплайне?

Подсказка: Подумайте о параллельных запросах и сокращении числа обращений к серверу.

Ответ:

- **Использование батчевых запросов к Redis**
- **Проверка кэша перед каждым gRPC запросом**
- **Асинхронное чтение данных для уменьшения блокировки**
- Сохранение всех данных без TTL

🔗 Шаг 10:

Вопрос 10 : Расположите шаги работы клиента с Redis и gRPC при сложной выборке данных:

1. Проверка нескольких ключей в Redis (батч)
2. Для отсутствующих ключей запрос к gRPC серверу
3. Сохранение новых данных в Redis с TTL
4. Обработка и объединение данных для клиента
5. Возврат объединенного результата клиенту

🔗 **Подсказка:**

Используйте кэш там, где данные есть, для остальных делаем запрос к серверу, сохраняем и собираем результат.

Ответ:

- Проверка нескольких ключей в Redis (батч)
- Для отсутствующих ключей запрос к gRPC серверу
- Сохранение новых данных в Redis с TTL
- Обработка и объединение данных для клиента
- Возврат объединенного результата клиенту

📖 Урок 3 - Сравнение задержек «прямой обработки» vs «кэшированный ответ»

🔗 Шаг 01:

Вопрос 1

Что происходит, когда клиент получает данные напрямую с сервера без кэша?

Подсказка: Подумайте о времени отклика.

Ответ:

- **Задержка выше, т.к. сервер обрабатывает запрос заново**
- Данные приходят мгновенно
- Клиент автоматически использует кэш
- Запрос идет в Kafka вместо сервера

🔗 Шаг 02:

Вопрос 2

Какие преимущества использования кэшированного ответа Redis?

Подсказка: Подумайте о скорости и нагрузке на сервер.

Ответ:

- **Уменьшение задержки отклика для клиента**
- **Снижение нагрузки на сервер**
- Автоматическое обновление данных на сервере
- **Возможность обработки нескольких запросов одновременно**

🔗 Шаг 03:

Вопрос 3: Расположите шаги сравнения задержек прямой обработки и кэшированного ответа:

1. Отправка запроса напрямую на сервер
2. Измерение времени отклика
3. Отправка запроса через кэш Redis
4. Измерение времени отклика и сравнение с прямой обработкой

🔗 Подсказка:

Сначала делаем прямую обработку, потом кэшированную, и сравниваем задержки.

Ответ:

- Отправка запроса напрямую на сервер
- Измерение времени отклика
- Отправка запроса через кэш Redis
- Измерение времени отклика и сравнение с прямой обработкой

🔗 Шаг 04:

Вопрос 5

Что важно учитывать при измерении задержек кэшированного и прямого запроса?

Подсказка: Подумайте о сетевой латентности и размере данных.

Ответ:

- **Сетевую латентность между клиентом и сервером**
- **Размер передаваемых данных**
- Тип используемого Kafka топика
- **Наличие других параллельных запросов**

🔗 Шаг 05:

Вопрос 6: Расположите шаги оптимизации работы с кэшем для снижения задержки:

1. Проверка наличия данных в Redis
2. Использование батчевых запросов к Redis
3. Запрос недостающих данных с сервера
4. Сохранение новых данных в Redis с TTL
5. Обработка и объединение данных для клиента

🔗 Подсказка:

Сначала проверяем кэш, затем оптимизируем запросы и сохраняем результаты.

Ответ:

- Проверка наличия данных в Redis
- Использование батчевых запросов к Redis
- Запрос недостающих данных с сервера

- Сохранение новых данных в Redis с TTL
- Обработка и объединение данных для клиента

Шаг 06:

Вопрос 7

Какие ситуации могут снизить преимущества кэширования в Redis?

Подсказка: Подумайте о свежести данных и отказах.

Ответ:

- **Данные быстро устаревают и TTL слишком большой**
- **Redis недоступен из-за сетевых проблем**
- Данные всегда берутся из Kafka, минуя Redis
- **Постоянные записи/удаления в кэше приводят к нагрузке**

Шаг 07:

Вопрос 8

Сопоставьте тип запроса с его ожидаемой задержкой:

Ответ:

- Прямой gRPC запрос без кэша
- Запрос кэшированного ответа в Redis
- Кэшированного ответа нет, идет на сервер
- Массив батчевых кэшированных запросов

1. Высокая задержка
2. Очень низкая задержка
3. Средняя задержка
4. Низкая задержка с экономией запросов

Подсказка:

Подумайте о кэш-хит/кэш-мисс и о батчевых оптимизациях.

- **Прямой gRPC запрос без кэша** — Высокая задержка
- **Запрос кэшированного ответа в Redis** — Очень низкая задержка
- **Кэшированного ответа нет, идет на сервер** — Средняя задержка
- **Массив батчевых кэшированных запросов** — Низкая задержка с экономией запросов

Шаг 08:

Вопрос 9

Что помогает уменьшить задержку при множественных параллельных запросах к Redis?

Подсказка: Подумайте о пакетной обработке и асинхронности.

Ответ:

- **Батчевые запросы к Redis**
- **Асинхронная обработка ответов**
- Сохранение всех данных без TTL
- **Проверка кэша перед gRPC запросом**

Шаг 09:

Вопрос 10 : Расположите шаги сравнения задержек прямой обработки и кэширования для анализа производительности:

1. Отправка нескольких прямых запросов к серверу
2. Измерение времени отклика каждого запроса
3. Отправка тех же запросов через кэш Redis
4. Измерение времени отклика кэшированных запросов
5. Сравнение и визуализация результатов

Подсказка:

Сначала тестируем прямую обработку, затем кэш, после сравниваем.

Ответ:

- Отправка нескольких прямых запросов к серверу
- Измерение времени отклика каждого запроса
- Отправка тех же запросов через кэш Redis
- Измерение времени отклика кэшированных запросов
- Сравнение и визуализация результатов

Урок 4 - Практика клиент, который запрашивает обработанный заказ

Шаг 01:

Вопрос 1

Что происходит, когда клиент отправляет запрос на обработку заказа напрямую на сервер?

Подсказка: Подумайте о времени отклика и нагрузке на сервер.

Ответ:

- **Задержка выше, сервер выполняет обработку запроса заново**
- Ответ приходит мгновенно
- Данные автоматически берутся из кэша
- Запрос отправляется в Kafka вместо сервера

Шаг 02:

Вопрос 2

Какие преимущества получает клиент, когда заказ обрабатывается и кэшируется в Redis?

Подсказка: Подумайте о скорости и нагрузке на сервер.

Ответ:

- **Быстрый доступ к результату заказа**
- **Снижение нагрузки на сервер**
- Автоматическая модификация данных заказа
- **Обработка нескольких запросов одновременно**

Шаг 03:

Вопрос 3: Расположите шаги обработки заказа с использованием кэша:

1. Клиент отправляет запрос
2. Проверка наличия данных в Redis
3. Если данных нет, запрос идет на сервер

4. Сервер обрабатывает заказ
5. Сохранение результата в Redis и отправка клиенту

 Подсказка:

Сначала проверяем кэш, затем обрабатываем на сервере при необходимости и сохраняем результат.

Ответ:

- Клиент отправляет запрос
- Проверка наличия данных в Redis
- Если данных нет, запрос идет на сервер
- Сервер обрабатывает заказ
- Сохранение результата в Redis и отправка клиенту

 Шаг 04:

Вопрос 5

Что важно учитывать при измерении задержки обработки заказа с кэшем и без него?

Подсказка: Подумайте о сетевой латентности и размере данных.

Ответ:

- **Сетевую латентность между клиентом и сервером**
- **Размер данных заказа**
- Тип Kafka топика
- **Количество параллельных запросов**

 Шаг 05:

Вопрос 6: Расположите шаги оптимизации работы клиента с кэшем Redis:

1. Проверка кэша перед отправкой запроса
2. Использование батчевых запросов для нескольких заказов
3. Запрос отсутствующих данных на сервер
4. Сохранение новых данных с TTL в Redis
5. Объединение и отправка ответа клиенту

 Подсказка:

Сначала проверяем кэш, затем обрабатываем недостающие данные, сохраняем результат и возвращаем клиенту.

Ответ:

- Проверка кэша перед отправкой запроса
- Использование батчевых запросов для нескольких заказов
- Запрос отсутствующих данных на сервер
- Сохранение новых данных с TTL в Redis
- Объединение и отправка ответа клиенту

 Шаг 06:

Вопрос 7

Какие факторы могут снизить преимущества кэширования обработанных заказов?

Подсказка: Подумайте о свежести данных и отказах кэша.

Ответ:

- **Данные быстро устаревают, TTL слишком большой**
- **Redis недоступен из-за сетевых проблем**
- Данные всегда берутся из Kafka, минуя Redis
- **Частые записи и удаления в кэше приводят к нагрузке**

 **Шаг 07:**

 **Вопрос 8**

Сопоставьте тип запроса клиента с ожидаемой задержкой:

Ответ:

- Прямой gRPC запрос без кэша
 - Запрос кэшированного заказа
 - Заказ не найден в кэше
 - Батчевые кэшированные запросы
1. Высокая задержка
 2. Очень низкая задержка
 3. Средняя задержка
 4. Низкая задержка с экономией запросов

 **Подсказка:**

Подумайте о кэш-хит/кэш-мисс и оптимизации батчами.

- **Прямой gRPC запрос без кэша** — Высокая задержка
- **Запрос кэшированного заказа** — Очень низкая задержка
- **Заказ не найден в кэше** — Средняя задержка
- **Батчевые кэшированные запросы** — Низкая задержка с экономией запросов

 **Шаг 08:**

Вопрос 9

Что помогает уменьшить задержку при множественных параллельных запросах к Redis?

Подсказка: Подумайте о пакетной обработке и асинхронности.

Ответ:

- **Батчевые запросы к Redis**
- **Асинхронная обработка ответов**
- Сохранение всех данных без TTL
- **Проверка кэша перед gRPC запросом**

 **Шаг 09:**

Вопрос 10 : Расположите шаги сравнения задержек обработки заказа и кэширования для анализа производительности:

1. Отправка нескольких прямых запросов к серверу
2. Измерение времени отклика каждого запроса
3. Отправка тех же запросов через кэш Redis
4. Измерение времени отклика кэшированных запросов
5. Сравнение и визуализация результатов

 **Подсказка:**

Сначала тестируем прямую обработку, затем кэш, после сравниваем.

Ответ:

- Отправка нескольких прямых запросов к серверу
- Измерение времени отклика каждого запроса
- Отправка тех же запросов через кэш Redis
- Измерение времени отклика кэшированных запросов
- Сравнение и визуализация результатов

Модуль 7 - Обеспечение надежности и масштабирования

Урок 1 - Обработка ошибок и повторные попытки (retry)

Шаг 01:

Вопрос 1

Какая стратегия чаще всего используется для обработки временных сетевых ошибок при отправке сообщений в Kafka?

Подсказка: Иногда сервисы временно недоступны и нужно повторить попытку.

Ответ:

- **Повторная отправка сообщения через определённый интервал**
- Игнорирование ошибки
- Удаление топика

Шаг 02:

Вопрос 2

Что лучше использовать для гарантированного порядка обработки сообщений в Kafka?

Подсказка: Подумайте о ключах и партициях.

Ответ:

- **Отправлять сообщения с одинаковым ключом в одну партицию**
- Использовать разные топика для каждого сообщения
- Отправлять случайно распределённо по партициям

Шаг 03:

Вопрос 3: Расположите шаги обработки ошибки при отправке события через gRPC в правильном порядке:

1. Обнаружение ошибки отправки
2. Логирование ошибки для мониторинга
3. Повторная попытка с экспоненциальным бэкоффом
4. Уведомление команды, если все попытки исчерпаны

Подсказка:

Сначала фиксируем ошибку, затем пробуем повторно, логируем и уведомляем команду при критических сбоях.

Ответ:

- Обнаружение ошибки отправки
- Логирование ошибки для мониторинга
- Повторная попытка с экспоненциальным бэкоффом

- Уведомление команды, если все попытки исчерпаны

🔗 Шаг 04:

Вопрос 4

Какая техника в Redis позволяет избежать потери данных при сбое потребителя очереди?

Подсказка: Используется механизм подтверждения обработки сообщений.

Ответ:

- **Использование списков с acknowledge (RPOPLPUSH + подтверждение)**
- Простое удаление элемента из списка
- Кэширование всех сообщений в памяти без подтверждения

🔗 Шаг 05:

📖 Вопрос 5: Сопоставление понятий

Сопоставьте каждый термин с правильным объяснением:

Понятия:

Ответ:

- gRPC
- Kafka
- Redis
- Retry

Определения:

1. Высокопроизводительный брокер сообщений для обработки потоков данных.
2. База данных с возможностью работы как брокер очередей и кэш.
3. Механизм повторной попытки при временных сбоях.
4. Фреймворк для удалённых вызовов с поддержкой протокола HTTP/2 и protobuf.

🔗 Подсказка:

Подумайте, где используется очередь, где кэш, а где сетевые вызовы.

- **gRPC** — Фреймворк для удалённых вызовов с поддержкой протокола HTTP/2 и protobuf.
- **Kafka** — Высокопроизводительный брокер сообщений для обработки потоков данных.
- **Redis** — База данных с возможностью работы как брокер очередей и кэш.
- **Retry** — Механизм повторной попытки при временных сбоях.

🔗 Шаг 06:

Вопрос 6

Какая стратегия масштабирования Kafka позволяет обработать большее количество сообщений без потери порядка внутри ключа?

Подсказка: Используйте несколько потребителей и партиций с ключами.

Ответ:

- **Добавление партиций и распределение сообщений по ключам**
- Один потребитель на топик
- Игнорирование партиций и ключей

Шаг 07:

Вопрос 7: Расположите шаги обработки ошибки при работе с Redis очередью в правильном порядке:

1. Попытка забрать сообщение из списка
2. В случае ошибки, вернуть сообщение в очередь и повторить через backoff
3. Логирование ошибки
4. Подтверждение успешной обработки сообщения

Подсказка:

Сначала берем сообщение, затем пытаемся обработать, логируем и подтверждаем.

Ответ:

- Попытка забрать сообщение из списка
- В случае ошибки, вернуть сообщение в очередь и повторить через backoff
- Логирование ошибки
- Подтверждение успешной обработки сообщения

Шаг 08:

Вопрос 8

Что из перечисленного помогает сделать пайплайн более устойчивым к кратковременным сбоям сети и сервисов?

Подсказка: Используем повторные попытки, таймауты и очереди.

Ответ:

- **Повторные попытки с экспоненциальным бэкоффом**
- **Буферизация сообщений в очередях**
- Игнорирование ошибок сети
- **Установка разумных таймаутов**

Шаг 09:

Вопрос 9: Сопоставление понятий масштабирования

Сопоставьте каждый термин с правильным объяснением:

Ответ:

- Шардирование
 - Репликация
 - Load balancing
 - Backpressure
1. Разделение данных на несколько частей для параллельной обработки.
 2. Создание копий данных для отказоустойчивости.
 3. Распределение нагрузки между узлами.
 4. Контроль скорости поступления данных, чтобы не перегружать сервис.

Подсказка:

Подумайте о масштабировании и надежности в реальном пайплайне.

- **Шардирование** — Разделение данных на несколько частей для параллельной обработки.
- **Репликация** — Создание копий данных для отказоустойчивости.
- **Load balancing** — Распределение нагрузки между узлами.

- **Backpressure** — Контроль скорости поступления данных, чтобы не перегружать сервис.

Шаг 10:

Вопрос 10

При проектировании масштабируемого event-driven пайплайна, что наиболее критично для обеспечения надежности и обработки больших объёмов сообщений?

Подсказка: Смотрите на брокеры сообщений, очереди и обработку ошибок.

Ответ:

- **Использование отказоустойчивых брокеров сообщений (Kafka, Redis)**
- **Реализация повторных попыток и backpressure**
- **Масштабирование потребителей и партиций**
- Хранение всех данных только в памяти одного сервиса

Урок 2 - Идемпотентность сообщений

Шаг 01:

Вопрос 1

Что означает идемпотентность сообщения в event-driven системах?

Подсказка: Подумайте о повторных вызовах и одинаковом результате.

Ответ:

- **Повторная обработка сообщения не изменяет результат**
- Каждое сообщение обязательно создаёт новый результат
- Сообщения должны обрабатываться только один раз

Шаг 02:

Вопрос 2

Какую технику используют, чтобы обеспечить идемпотентность в Kafka при повторной доставке сообщений?

Подсказка: Подумайте о ключах и идентификаторах сообщений.

Ответ:

- **Использование уникального идентификатора (message key)**
- Просто повторно отправлять сообщения без изменений
- Удалять топики после каждой обработки

Шаг 03:

Вопрос 3: Расположите шаги обработки повторного сообщения в правильном порядке:

1. Проверка идентификатора сообщения
2. **Если сообщение уже обработано** — игнорировать
3. Обработка сообщения
4. Сохранение статуса обработки

Подсказка:

Сначала проверяем, потом обрабатываем и сохраняем, чтобы избежать повторов.

Ответ:

- Проверка идентификатора сообщения

- **Если сообщение уже обработано** — игнорировать
- Обработка сообщения
- Сохранение статуса обработки

🔗 Шаг 04:

Вопрос 4

Почему важно делать обработку сообщений идемпотентной при масштабировании потребителей?

Подсказка: Подумайте о параллельной обработке и дубликатах.

Ответ:

- **Чтобы несколько потребителей могли обрабатывать одно и то же сообщение без ошибок**
- Чтобы ускорить отправку сообщений
- Чтобы уменьшить размер топика

🔗 Шаг 05:

📄 Вопрос 5: Сопоставление понятий

Сопоставьте термин с правильным объяснением:

Ответ:

- Идемпотентность
 - Message Key
 - Offset
 - Duplicate
1. Уникальный идентификатор для определения повторов
 2. Положение сообщения в топике для отслеживания прогресса
 3. Повторная обработка не меняет результат
 4. Сообщение, которое пришло повторно

🔗 Подсказка:

Подумайте о том, как система отслеживает обработку и предотвращает дубли.

- **Идемпотентность** — Повторная обработка не меняет результат
- **Message Key** — Уникальный идентификатор для определения повторов
- **Offset** — Положение сообщения в топике для отслеживания прогресса
- **Duplicate** — Сообщение, которое пришло повторно

🔗 Шаг 06:

Вопрос 6

Какую стратегию лучше использовать для идемпотентной обработки сообщений в Redis Queue?

Подсказка: Подумайте о сохранении статуса выполнения и подтверждении обработки.

Ответ:

- **RPOPLPUSH с проверкой статуса и подтверждением**
- RPOP без проверки статуса
- Сохранение всех сообщений в памяти без подтверждения

Шаг 07:

Вопрос 7: Расположите шаги обработки повторного сообщения с использованием идемпотентного подхода:

1. Получение сообщения из топика
2. Проверка, было ли оно уже обработано
3. Сохранение результата обработки
4. Подтверждение успешной обработки, чтобы удалить из очереди

Подсказка:

Сначала проверяем, потом обрабатываем и сохраняем прогресс, чтобы избежать повторов.

Ответ:

- Получение сообщения из топика
- Проверка, было ли оно уже обработано
- Сохранение результата обработки
- Подтверждение успешной обработки, чтобы удалить из очереди

Шаг 08:

Вопрос 8

Что из перечисленного помогает сделать пайплайн устойчивым к дубликатам сообщений?

Подсказка: Используем идентификаторы и проверку статуса обработки.

Ответ:

- **Использование уникального идентификатора сообщения**
- **Проверка, было ли сообщение обработано**
- Просто игнорирование дубликатов без проверки
- **Сохранение статуса обработки в БД или Redis**

Шаг 09:

Вопрос 9: Сопоставление концепций надежности

Сопоставьте термин с объяснением:

Ответ:

- Идемпотентность
 - Exactly-once
 - At-least-once
 - At-most-once
1. Сообщение может быть доставлено один раз или несколько раз, результат одинаковый
 2. Гарантия, что сообщение будет обработано ровно один раз
 3. Гарантия, что сообщение будет обработано хотя бы один раз, возможны дубликаты
 4. Сообщение может быть потеряно, но не будет дубликатов

Подсказка:

Подумайте о различии между дубликатами и потерями сообщений.

- **Идемпотентность** — Сообщение может быть доставлено один раз или несколько раз, результат одинаковый
- **Exactly-once** — Гарантия, что сообщение будет обработано ровно один раз

- **At-least-once** — Гарантия, что сообщение будет обработано хотя бы один раз, возможны дубликаты
- **At-most-once** — Сообщение может быть потеряно, но не будет дубликатов

🔔 Шаг 10:

Вопрос 10

Что является критическим для обеспечения надежного масштабируемого пайплайна с идемпотентной обработкой сообщений?

Подсказка: Смотрите на брокеры сообщений, контроль дубликатов и обработку ошибок.

Ответ:

- **Использование брокеров с подтверждением доставки (Kafka, Redis)**
- **Идемпотентная обработка с проверкой статуса сообщений**
- **Повторные попытки и контроль backpressure**
- Игнорирование дубликатов и ошибок

📖 Урок 3 - Масштабирование Kafka Consumers

🔔 Шаг 01:

Вопрос 1

Что происходит при увеличении числа Kafka Consumers в одной группе?

Подсказка: Подумайте о распределении партиций.

Ответ:

- **Партиции распределяются между потребителями группы**
- Все потребители читают все партиции одновременно
- Новые потребители игнорируются брокером

🔔 Шаг 02:

Вопрос 2

Какой ограничивающий фактор существует при масштабировании потребителей Kafka?

Подсказка: Подумайте о количестве партиций топика.

Ответ:

- **Количество партиций топика ограничивает число активных потребителей**
- Количество брокеров не влияет на масштабирование
- Количество потребителей не имеет ограничений

🔔 Шаг 03:

Вопрос 3: Расположите шаги при добавлении нового Kafka Consumer в группу:

1. Consumer соединяется с брокером
2. Брокер выполняет ребалансировку группы
3. Consumer начинает получать данные из выделенных партиций
4. Обновление оффсетов и состояния партиций

🔔 Подсказка:

Сначала подключение, потом ребалансировка, затем обработка и обновление состояния.

Ответ:

- Consumer соединяется с брокером
- Брокер выполняет ребалансировку группы
- Consumer начинает получать данные из выделенных партиций
- Обновление оффсетов и состояния партиций

Шаг 04:

Вопрос 4

Почему важно контролировать количество потребителей на топик при масштабировании?

Подсказка: Подумайте о неэффективном распределении партиций.

Ответ:

- **Чтобы каждый потребитель имел хотя бы одну партицию для обработки**
- Чтобы уменьшить нагрузку на брокеры до нуля
- Чтобы потребители не конкурировали за оффсеты

Шаг 05:

Вопрос 5: Сопоставление понятий масштабирования Kafka

Сопоставьте термин с его объяснением:

Ответ:

- Consumer Group
 - Partition
 - Rebalance
 - Offset
1. Набор потребителей, совместно обрабатывающих топик
 2. Единица параллельной обработки данных в топике
 3. Процесс перераспределения партиций между потребителями
 4. Позиция в партиции для отслеживания прогресса

Подсказка:

Подумайте о том, как брокер распределяет данные между потребителями и следит за прогрессом.

- **Consumer Group** — Набор потребителей, совместно обрабатывающих топик
- **Partition** — Единица параллельной обработки данных в топике
- **Rebalance** — Процесс перераспределения партиций между потребителями
- **Offset** — Позиция в партиции для отслеживания прогресса

Шаг 06:

Вопрос 6

Что происходит с потребителем, когда топик расширяется новыми партициями?

Подсказка: Подумайте о перераспределении нагрузки.

Ответ:

- **Проходит ребалансировка, и потребители получают новые партиции**
- Новые партиции игнорируются автоматически
- Брокер удаляет старые партиции

Шаг 07:

Вопрос 7: Расположите шаги при масштабировании потребителей в правильном порядке:

1. Добавление нового потребителя
2. Consumer соединяется с брокером
3. Ребалансировка группы
4. Обработка сообщений с новых партиций

Подсказка:

Сначала добавление, затем подключение, ребалансировка, обработка.

Ответ:

- Добавление нового потребителя
- Consumer соединяется с брокером
- Ребалансировка группы
- Обработка сообщений с новых партиций

Шаг 08:

Вопрос 8

Почему важно следить за балансом нагрузки между потребителями?

Подсказка: Подумайте о времени обработки и задержках.

Ответ:

- Чтобы ни один потребитель не был перегружен
- Чтобы обрабатывать сообщения равномерно и быстро
- Чтобы один потребитель мог игнорировать часть партиций

Шаг 09:

Вопрос 9: Сопоставление стратегий масштабирования

Сопоставьте стратегию с её особенностью:

Ответ:

- Horizontal scaling
 - Vertical scaling
 - Rebalance
 - Partition expansion
1. Добавление новых потребителей для параллельной обработки
 2. Увеличение ресурсов одного потребителя
 3. Перераспределение партиций после изменения числа потребителей
 4. Увеличение числа партиций топика для большей параллельности

Подсказка:

Подумайте, как изменяется масштабирование и распределение нагрузки.

- **Horizontal scaling** — Добавление новых потребителей для параллельной обработки
- **Vertical scaling** — Увеличение ресурсов одного потребителя
- **Rebalance** — Перераспределение партиций после изменения числа потребителей
- **Partition expansion** — Увеличение числа партиций топика для большей параллельности

Шаг 10:

Вопрос 10

Что является критическим для надежного масштабирования Kafka Consumers в продакшене?

Подсказка: Подумайте о равномерности нагрузки, оффсетах и ребалансировках.

Ответ:

- **Балансировка нагрузки между потребителями**
- **Контроль оффсетов и прогресса обработки**
- **Корректная обработка ребалансировок без потерь данных**
- Игнорирование числа партиций топика

Урок 4 - Горизонтальное масштабирование сервисов

Шаг 01:

Вопрос 1

Что означает горизонтальное масштабирование сервисов?

Подсказка: Подумайте о добавлении новых экземпляров сервиса.

Ответ:

- **Добавление новых инстансов сервиса для распределения нагрузки**
- Увеличение ресурсов одного сервиса (CPU, RAM)
- Оптимизация кода без добавления новых серверов

Шаг 02:

Вопрос 2

Почему горизонтальное масштабирование предпочтительнее вертикального для распределенных систем?

Подсказка: Подумайте о надежности и отказоустойчивости.

Ответ:

- **Позволяет выдерживать отказ отдельных инстансов и легко добавлять новые**
- Требуется меньше сетевых ресурсов
- Снижает необходимость в балансировщиках нагрузки

Шаг 03:

Вопрос 3: Расположите шаги для горизонтального масштабирования сервиса:

1. Добавление новых экземпляров сервиса
2. Настройка балансировщика нагрузки
3. Проверка доступности новых инстансов
4. Равномерное распределение трафика между всеми инстансами

Подсказка:

Сначала добавляем, потом настраиваем балансировку, затем проверяем и распределяем нагрузку.

Ответ:

- Добавление новых экземпляров сервиса
- Настройка балансировщика нагрузки
- Проверка доступности новых инстансов

- Равномерное распределение трафика между всеми инстансами

🔗 Шаг 04:

Вопрос 4

Что нужно учитывать при горизонтальном масштабировании stateful сервисов?

Подсказка: Подумайте о состоянии и синхронизации данных между инстансами.

Ответ:

- **Синхронизацию состояния между инстансами**
- **Использование внешнего хранилища для общего состояния**
- Игнорирование состояния при масштабировании

🔗 Шаг 05:

📖 Вопрос 5: Сопоставление понятий горизонтального масштабирования

Сопоставьте термин с его объяснением:

Ответ:

- Load Balancer
 - Instance
 - Stateless Service
 - Stateful Service
1. Компонент, распределяющий трафик между экземплярами
 2. Отдельный запущенный процесс сервиса
 3. Сервис без сохранения состояния между запросами
 4. Сервис, сохраняющий состояние между запросами

🔗 Подсказка:

Подумайте, как балансировщик работает с различными типами сервисов и инстансами.

- **Load Balancer** — Компонент, распределяющий трафик между экземплярами
- **Instance** — Отдельный запущенный процесс сервиса
- **Stateless Service** — Сервис без сохранения состояния между запросами
- **Stateful Service** — Сервис, сохраняющий состояние между запросами

🔗 Шаг 06:

Вопрос 6

Что помогает равномерно распределять нагрузку между инстансами?

Подсказка: Подумайте о компонентах, которые принимают решения о маршрутизации.

Ответ:

- **Load Balancer**
- **DNS Round Robin**
- Только один инстанс без балансировки

🔗 Шаг 07:

Вопрос 7: Расположите действия при масштабировании stateless сервиса:

1. Развертывание нового инстанса
2. Обновление конфигурации балансировщика

3. Проверка доступности инстанса
4. Начало обработки запросов

 Подсказка:

Сначала создаем инстанс, затем настраиваем балансировку, проверяем и начинаем обработку.

Ответ:

- Развертывание нового инстанса
- Обновление конфигурации балансировщика
- Проверка доступности инстанса
- Начало обработки запросов

 Шаг 08:

Вопрос 8

Почему stateless сервис проще масштабировать, чем stateful?

Подсказка: Подумайте о хранении состояния и зависимости между запросами.

Ответ:

- **Не требует синхронизации состояния между инстансами**
- **Любой инстанс может обрабатывать любой запрос**
- Меньше инстансов нужно разворачивать

 Шаг 09:

 Вопрос 9: Сопоставление преимуществ горизонтального масштабирования

Сопоставьте преимущество с объяснением:

Ответ:

- Отказоустойчивость
 - Гибкость
 - Высокая производительность
 - Легкость обновлений
1. Система продолжает работать при падении отдельных инстансов
 2. Легко добавлять или удалять инстансы при изменении нагрузки
 3. Обработка большего числа запросов за счет параллельной обработки
 4. Новые версии можно внедрять без остановки всей системы

 Подсказка:

Подумайте, как горизонтальное масштабирование влияет на отказоустойчивость, нагрузку и обновления.

- **Отказоустойчивость** — Система продолжает работать при падении отдельных инстансов
- **Гибкость** — Легко добавлять или удалять инстансы при изменении нагрузки
- **Высокая производительность** — Обработка большего числа запросов за счет параллельной обработки
- **Легкость обновлений** — Новые версии можно внедрять без остановки всей системы

 Шаг 10:

Вопрос 10

Что критично для успешного горизонтального масштабирования в продакшене?

Подсказка: Подумайте о балансировке нагрузки, мониторинге и автоматизации.

Ответ:

- **Эффективный балансировщик нагрузки**
- **Автоматическое масштабирование инстансов**
- **Мониторинг состояния всех инстансов**
- Игнорирование состояния сервисов

Урок 5 - Практика симуляция сбоев и проверка устойчивости пайплайна.

Шаг 01:

Вопрос 1

Что является первым шагом при тестировании устойчивости пайплайна?

Подсказка: Начинаем с планирования сценариев сбоев.

Ответ:

- **Определение сценариев возможных сбоев**
- Запуск всех компонентов в продакшне
- Мониторинг метрик без тестирования

Шаг 02:

Вопрос 2

Какую роль выполняет механизм повторной отправки сообщений в Kafka при сбоях?

Подсказка: Подумайте, что делать, если один из потребителей не смог обработать сообщение.

Ответ:

- **Обеспечивает доставку сообщения даже после временного сбоя**
- Удаляет сообщения после первой попытки
- Создает резервный кластер автоматически

Шаг 03:

Вопрос 3

Сопоставьте концепции обеспечения надежности с их объяснением:

Ответ:

- Retry
- Idempotency
- Dead Letter Queue (DLQ)
- Monitoring

1. Повторное выполнение задачи без побочных эффектов
2. Запись сообщений, которые не удалось обработать
3. Автоматический повтор при временной ошибке
4. Слежение за состоянием системы и метриками

Подсказка: Подумайте, как система справляется с ошибками и контролирует состояние.

- **Retry** — Автоматический повтор при временной ошибке
- **Idempotency** — Повторное выполнение задачи без побочных эффектов
- **Dead Letter Queue (DLQ)** — Запись сообщений, которые не удалось обработать
- **Monitoring** — Слежение за состоянием системы и метриками

Шаг 04:

Вопрос 4

Что делает Redis в пайплайне при необходимости масштабирования потребителей?

Подсказка: Подумайте, как можно распределять задачи между несколькими воркерами.

Ответ:

- **Используется как очередь для распределения задач между воркерами**
- Хранит метрики производительности сервиса
- Автоматически создает новые темы Kafka

Шаг 05:

Вопрос 5 Расположите шаги при обработке сообщений с DLQ:

1. Сообщение не удалось обработать
2. Отправка сообщения в DLQ
3. Анализ причин ошибки
4. Попытка повторной обработки или исправления

Подсказка: DLQ помогает изолировать проблемные сообщения, чтобы основной поток не падал.

Ответ:

- Сообщение не удалось обработать
- Отправка сообщения в DLQ
- Анализ причин ошибки
- Попытка повторной обработки или исправления

Шаг 06:

Вопрос 6

Что помогает предотвратить повторную обработку одного и того же события в Kafka?

Подсказка: Подумайте, как сделать обработку событий безопасной при сбоях.

Ответ:

- **Idempotent consumer (идемпотентный потребитель)**
- Использование `bufcomp`
- Создание новых топиков

Шаг 07:

Вопрос 7

Сопоставьте термины масштабирования с их характеристикой:

Ответ:

- Horizontal Scaling
 - Vertical Scaling
 - Partitioning
 - Replication
1. Добавление новых серверов в кластер
 2. Увеличение ресурсов одного сервера
 3. Разделение данных между топиками или брокерами
 4. Создание копий данных для отказоустойчивости

Подсказка: Думайте, как система справляется с ростом нагрузки и отказами.

- **Horizontal Scaling** — Добавление новых серверов в кластер
- **Vertical Scaling** — Увеличение ресурсов одного сервера
- **Partitioning** — Разделение данных между топиками или брокерами
- **Replication** — Создание копий данных для отказоустойчивости

🔧 Шаг 08:

Вопрос 8 Расположите шаги при симуляции сбоев в пайплайне:

1. Определение сценариев сбоев (отказ сервиса, задержка сообщений)
2. Изоляция компонентов для тестирования
3. Запуск симуляции и сбор логов
4. Анализ и исправление проблем

Подсказка: Сначала планируем, потом проверяем, потом исправляем.

Ответ:

- Определение сценариев сбоев (отказ сервиса, задержка сообщений)
- Изоляция компонентов для тестирования
- Запуск симуляции и сбор логов
- Анализ и исправление проблем

🔧 Шаг 09:

Вопрос 9

Какой инструмент позволяет визуализировать задержки и ошибки в event-driven пайплайне на продакшн-сервере?

Подсказка: Обычно интегрируется с Prometheus или Grafana.

Ответ:

- **Prometheus + Grafana**
- bufconn
- Mock client

🔧 Шаг 10:

Вопрос 10

Какой подход повышает отказоустойчивость пайплайна при масштабировании Kafka и Redis?

Подсказка: Думайте о резервировании и распределении нагрузки.

Ответ:

- **Replication, Partitioning и Horizontal Scaling**
- Использование только одного брокера
- Отключение мониторинга

📁 Модуль 8 - Мониторинг и логирование

📖 Урок 1 - Метрики Kafka, gRPC и Redis

🔧 Шаг 01:

Вопрос 1

Какая метрика Kafka показывает количество сообщений, ожидающих обработки потребителем?

Подсказка: Подумайте о метриках задержки потребителя.

Ответ:

- **Consumer lag**
- CPU usage
- Disk I/O
- Memory usage

Шаг 02:

Вопрос 2

Какая метрика gRPC помогает оценить производительность сервиса?

Подсказка: Смотрите на количество запросов и время их обработки.

Ответ:

- **Requests per second (RPS)**
- Heap memory
- Disk latency
- Thread count

Шаг 03:

Вопрос 3 Расположите шаги мониторинга Redis в правильном порядке:

1. Сбор метрик (используя INFO)
2. Анализ ключевых показателей (latency, memory, hits/misses)
3. Настройка алертов
4. Визуализация в дашборде

 **Подсказка:**

Сначала собираем данные, затем анализируем, потом уведомляем и визуализируем.

Ответ:

- Сбор метрик (используя INFO)
- Анализ ключевых показателей (latency, memory, hits/misses)
- Настройка алертов
- Визуализация в дашборде

Шаг 04:

Вопрос 4

Сопоставьте метрику с её назначением:

Ответ:

- Consumer lag
 - Request latency
 - Memory usage
 - Cache hit ratio
1. Задержка обработки запроса
 2. Используемая память сервиса
 3. Количество сообщений, ожидающих обработки
 4. Эффективность кэширования

🔔 Подсказка:

Метрики отражают либо задержку, либо загрузку ресурсов.

- **Consumer lag** — Количество сообщений, ожидающих обработки
- **Request latency** — Задержка обработки запроса
- **Memory usage** — Используемая память сервиса
- **Cache hit ratio** — Эффективность кэширования

🔔 Шаг 05:

Вопрос 5

Какая комбинация метрик поможет понять, что Kafka кластер перегружен?

Подсказка: Смотрите на задержку и накопление сообщений.

Ответ:

- **Consumer lag**
- **Under-replicated partitions**
- Heap memory
- Disk latency

🔔 Шаг 06:

Вопрос 6 Расположите шаги диагностики gRPC сервиса:

1. Сбор метрик RPS и latency
2. Определение медленных эндпоинтов
3. Проверка ошибок и статусов ответов
4. Настройка алертов и дашбордов

🔔 Подсказка:

Сначала измеряем, затем анализируем, потом отслеживаем.

Ответ:

- Сбор метрик RPS и latency
- Определение медленных эндпоинтов
- Проверка ошибок и статусов ответов
- Настройка алертов и дашбордов

🔔 Шаг 07:

Вопрос 7

Какой инструмент удобно использовать для визуализации метрик Redis?

Подсказка: Популярный стек мониторинга с графиками и алертами.

Ответ:

- **Grafana**
- Postman
- VSCode
- Docker

🔔 Шаг 08:

Вопрос 8

Сопоставьте инструмент и тип данных, которые он показывает:

Ответ:

- Prometheus
- Grafana
- Redis INFO
- Kafka JMX

1. Метрики сервисов и кластеров
2. Визуализация и дашборды
3. Информация о состоянии Redis
4. Метрики брокеров Kafka

 **Подсказка:**

Подумайте о роли каждого инструмента: сбор, визуализация или локальная диагностика.

- **Prometheus** — Метрики сервисов и кластеров
- **Grafana** — Визуализация и дашборды
- **Redis INFO** — Информация о состоянии Redis
- **Kafka JMX** — Метрики брокеров Kafka

Урок 2 - Логирование событий и запросов

 **Шаг 01:**

Вопрос 1

Что является основным назначением логирования в сервисе?

Подсказка: Логи помогают понять, что происходит в приложении.

Ответ:

- **Отслеживать события и ошибки в приложении**
- Сохранять конфиденциальные пароли
- Ускорять работу сервиса
- Замедлять обработку запросов

 **Шаг 02:**

Вопрос 2

Какая информация чаще всего включается в лог-запись?

Подсказка: Думайте о времени, уровне и сообщении.

Ответ:

- **Время события, уровень логирования, сообщение**
- Пароли пользователей
- Код приложения полностью
- Резервные копии базы данных

 **Шаг 03:**

Вопрос 3 Расположите шаги анализа логов в правильном порядке:

1. Сбор логов с сервиса
2. Фильтрация по уровню и типу события
3. Анализ причин ошибок
4. Вывод отчета и рекомендации

🔗 Подсказка:

Сначала собираем, затем фильтруем, анализируем и выводим результаты.

Ответ:

- Сбор логов с сервиса
- Фильтрация по уровню и типу события
- Анализ причин ошибок
- Вывод отчета и рекомендации

🔗 Шаг 04:

Вопрос 4

Сопоставьте уровень логирования с его назначением:

Ответ:

- INFO
 - DEBUG
 - WARN
 - ERROR
1. Отображение нормальных операций и событий
 2. Детальная информация для разработки и отладки
 3. Предупреждение о потенциальной проблеме
 4. Ошибки, требующие немедленного внимания

🔗 Подсказка:

Уровни отражают важность и цель сообщения.

- **INFO** — Отображение нормальных операций и событий
- **DEBUG** — Детальная информация для разработки и отладки
- **WARN** — Предупреждение о потенциальной проблеме
- **ERROR** — Ошибки, требующие немедленного внимания

🔗 Шаг 05:

Вопрос 5

Какой инструмент чаще всего используется для централизованного хранения логов?

Подсказка: Популярный стек ELK или EFK.

Ответ:

- **Elasticsearch + Logstash + Kibana**
- MySQL
- PostgreSQL
- Redis

🔗 Шаг 06:

Вопрос 6 Расположите шаги обработки логов для выявления проблем:

1. Централизованный сбор логов
2. Парсинг и структурирование
3. Фильтрация и поиск аномалий
4. Уведомление и алерты

💡 Подсказка:

Сначала собираем, затем обрабатываем и реагируем.

Ответ:

- Централизованный сбор логов
- Парсинг и структурирование
- Фильтрация и поиск аномалий
- Уведомление и алерты

💡 Шаг 07:

Вопрос 7

Почему важно не логировать конфиденциальные данные?

Подсказка: Безопасность и соответствие требованиям GDPR/PCI.

Ответ:

- **Чтобы предотвратить утечку личных данных**
- Чтобы ускорить приложение
- Чтобы уменьшить объем диска
- Чтобы увеличить нагрузку на сеть

💡 Шаг 08:

Вопрос 8

Сопоставьте инструмент с его ролью в логировании:

Ответ:

- Logstash
 - Kibana
 - Filebeat
 - Elasticsearch
1. Парсинг и отправка логов
 2. Агрегация и хранение
 3. Сбор логов с серверов
 4. Визуализация и дашборды

💡 Подсказка:

Каждый компонент имеет свою функцию: сбор, хранение, обработка или визуализация.

- **Logstash** — Парсинг и отправка логов
- **Kibana** — Визуализация и дашборды
- **Filebeat** — Сбор логов с серверов
- **Elasticsearch** — Агрегация и хранение

📖 Урок 3 - Настройка Prometheus + Grafana (базово)

💡 Шаг 01:

Вопрос 1

Зачем настраивать Prometheus в микросервисном приложении на gRPC?

Подсказка: Подумайте о видимости работы сервисов.

Ответ:

- **Чтобы собирать метрики производительности и ошибок**
- Чтобы автоматически деплоить сервисы
- Чтобы хранить логи в базе данных
- Чтобы заменить Redis для кэша

🔗 Шаг 02:

Вопрос 2

Какие типы метрик чаще всего собирают с gRPC сервисов через Prometheus?

Подсказка: Вспомните задержки и количество запросов.

Ответ:

- **Latency (задержки)**
- **Request count (число запросов)**
- Файловые дескрипторы
- **Error rate (частота ошибок)**

🔗 Шаг 03:

Вопрос 3: Расположите шаги подключения Prometheus к gRPC сервису:

1. Добавить экспортер метрик в код сервиса
2. Настроить эндпоинт /metrics
3. Добавить сервис в конфигурацию Prometheus
4. Проверить сбор метрик в интерфейсе Prometheus

🔗 Подсказка:

Сначала добавляем экспортер, затем настраиваем метрики, регистрируем сервис и проверяем результат.

Ответ:

- Добавить экспортер метрик в код сервиса
- Настроить эндпоинт /metrics
- Добавить сервис в конфигурацию Prometheus
- Проверить сбор метрик в интерфейсе Prometheus

🔗 Шаг 04:

Вопрос 4

Сопоставьте метрику с её назначением:

Ответ:

- Histogram
 - Counter
 - Gauge
 - Summary
1. Счётчик, который только увеличивается
 2. Метрика текущего состояния (температура, память)
 3. Распределение значений (например, задержки)
 4. Агрегированная статистика по выборке

🔔 Подсказка:

Вспомните: counter только растёт, gauge может уменьшаться, histogram собирает распределение.

- **Histogram** — Распределение значений (например, задержки)
- **Counter** — Счётчик, который только увеличивается
- **Gauge** — Метрика текущего состояния (температура, память)
- **Summary** — Агрегированная статистика по выборке

🔔 Шаг 05:

Вопрос 5

Почему важно настраивать алерты в Prometheus?

Подсказка: Вспомните про автоматическое оповещение DevOps.

Ответ:

- **Чтобы получать уведомления о проблемах до того, как они станут критичными**
- Чтобы ускорить компиляцию сервисов
- Чтобы уменьшить количество логов
- Чтобы заменить Kafka

🔔 Шаг 06:

Вопрос 6: Расположите шаги настройки дашборда в Grafana:

1. Подключить источник данных Prometheus
2. Создать новый дашборд
3. Добавить панель с запросом к метрике
4. Сохранить и поделиться дашбордом

🔔 Подсказка:

Сначала подключаем источник данных, затем создаём дашборд и панели, и в конце сохраняем.

Ответ:

- Подключить источник данных Prometheus
- Создать новый дашборд
- Добавить панель с запросом к метрике
- Сохранить и поделиться дашбордом

🔔 Шаг 07:

Вопрос 7

Сопоставьте инструмент с его функцией:

Ответ:

- Prometheus
 - Grafana
 - Alertmanager
 - Loki
1. Визуализация данных и создание дашбордов
 2. Сбор и хранение метрик
 3. Оповещения при нарушении правил
 4. Сбор и хранение логов

Подсказка:

В связке Prometheus+Grafana+Alertmanager+Loki каждый отвечает за своё.

- **Prometheus** — Сбор и хранение метрик
- **Grafana** — Визуализация данных и создание дашбордов
- **Alertmanager** — Оповещения при нарушении правил
- **Loki** — Сбор и хранение логов

Шаг 08:

Вопрос 8

Какая комбинация инструментов позволит построить полный цикл мониторинга и логирования?

Подсказка: Подумайте о метриках, визуализации и логах.

Ответ:

- **Prometheus + Grafana + Alertmanager + Loki**
- Kafka + Redis
- **Prometheus + Grafana**
- Redis + gRPC

Шаг 09:

Вопрос 9: Расположите шаги обработки инцидента при падении сервиса:

1. Alertmanager отправляет уведомление
2. DevOps получает уведомление
3. Открывается дашборд Grafana для анализа
4. Смотрим логи в Loki для деталей
5. Фиксим проблему и проверяем метрики

Подсказка:

Сначала приходит оповещение, затем анализ и устранение.

Ответ:

- Alertmanager отправляет уведомление
- DevOps получает уведомление
- Открывается дашборд Grafana для анализа
- Смотрим логи в Loki для деталей
- Фиксим проблему и проверяем метрики

Урок 4 - Практика добавить метрики и логи в сервисы.

Шаг 01:

Вопрос 1

Какой инструмент лучше всего использовать для сбора метрик из gRPC-сервисов?

Подсказка: Ищем инструмент, который умеет собирать метрики с endpoint и строить графики.

Ответ:

- **Prometheus**
- Redis
- Kafka

🔗 Шаг 02:

Вопрос 2

Какой тип логирования помогает быстро обнаружить ошибки при вызове gRPC методов?

Подсказка: Логи должны содержать информацию о статусе вызова и ошибках.

Ответ:

- **Error log**
- Debug log без контекста
- Информационный log без уровня

🔗 Шаг 03:

Вопрос 3 Расположите шаги по добавлению метрик в gRPC сервис в правильном порядке:

1. Добавление endpoint для Prometheus
2. Определение нужных метрик (счетчики, гистограммы)
3. Инструментирование вызовов gRPC для записи метрик
4. Проверка, что метрики собираются корректно

🔗 **Подсказка:**

Сначала определяем, что измеряем, потом подключаем метрики к вызовам, создаем endpoint и проверяем сбор данных.

Ответ:

- Определение нужных метрик (счетчики, гистограммы)
- Инструментирование вызовов gRPC для записи метрик
- Добавление endpoint для Prometheus
- Проверка, что метрики собираются корректно

🔗 Шаг 04:

Вопрос 4

Сопоставьте типы метрик с их назначением:

Ответ:

- Counter
 - Gauge
 - Histogram
 - Summary
1. Измеряет количество событий за время
 2. Отслеживает текущее значение переменной
 3. Собирает распределение значений (например, время отклика)
 4. Позволяет агрегировать выборки для анализа перцентилей

🔗 **Подсказка:**

Подумайте, что нужно для подсчета событий, отслеживания состояния и анализа латентности.

- **Counter** — Измеряет количество событий за время
- **Gauge** — Отслеживает текущее значение переменной
- **Histogram** — Собирает распределение значений (например, время отклика)
- **Summary** — Позволяет агрегировать выборки для анализа перцентилей

💡 Шаг 05:

Вопрос 5

Какой подход поможет минимизировать потерю логов при высоких нагрузках в Kafka?

Подсказка: Нужно думать о надежной доставке сообщений.

Ответ:

- **Использовать подтверждения (acks) и ретраи**
- Просто писать в один топик без настройки
- Сохранять логи только в памяти

💡 Шаг 06:

Вопрос 6 Расположите шаги настройки логирования в Redis Streams:

1. Создать consumer группу для логов
2. Отправка логов в поток
3. Обработка и сохранение логов
4. Мониторинг задержек и ошибок

💡 **Подсказка:**

Сначала создаем поток и группу, затем логируем, обрабатываем и мониторим.

Ответ:

- Создать consumer группу для логов
- Отправка логов в поток
- Обработка и сохранение логов
- Мониторинг задержек и ошибок

💡 Шаг 07:

Вопрос 7

Сопоставьте типы логов с примерами:

Ответ:

- Info
 - Warning
 - Error
 - Debug
1. Проблема при подключении к базе, но работа продолжается
 2. Старт сервиса и успешная инициализация
 3. Детальная информация о вызовах методов для разработчиков
 4. Сбой метода gRPC с выбросом исключения

💡 **Подсказка:**

Info — это обычные события, Warning — предупреждения, Error — ошибки, Debug — подробная отладка.

- **Info** — Старт сервиса и успешная инициализация
- **Warning** — Проблема при подключении к базе, но работа продолжается
- **Error** — Сбой метода gRPC с выбросом исключения
- **Debug** — Детальная информация о вызовах методов для разработчиков

🔗 Шаг 08:

Вопрос 8

Что поможет оптимизировать производительность сбора метрик при большом числе gRPC вызовов?

Подсказка: Думайте о пакетной отправке данных и снижении нагрузки.

Ответ:

- **Использовать batch экспорт метрик**
- **Инструментировать только ключевые методы**
- Отправлять метрики при каждом вызове без фильтрации

🔗 Шаг 09:

Вопрос 9 Расположите действия по обнаружению и устранению медленных gRPC вызовов:

1. Сбор метрик времени отклика
2. Идентификация методов с высокой задержкой
3. Оптимизация кода и запросов
4. Перепроверка после изменений

🔗 **Подсказка:**

Сначала измеряем, потом анализируем, оптимизируем и проверяем эффект.

Ответ:

- Сбор метрик времени отклика
- Идентификация методов с высокой задержкой
- Оптимизация кода и запросов
- Перепроверка после изменений

🔗 Шаг 10:

Вопрос 10

Сопоставьте инструменты с их функциями в пайплайне мониторинга:

Ответ:

- Prometheus
 - Grafana
 - Kafka
 - Redis
1. Визуализация метрик и дашборды
 2. Сбор и хранение метрик
 3. Буферизация и передача логов/событий
 4. Очередь событий для обработки и мониторинга потоков

🔗 **Подсказка:**

Prometheus собирает, Grafana показывает, Kafka и Redis помогают с потоками и логами.

- **Prometheus** — Сбор и хранение метрик
- **Grafana** — Визуализация метрик и дашборды
- **Kafka** — Буферизация и передача логов/событий
- **Redis** — Очередь событий для обработки и мониторинга потоков

Модуль 9 - CI/CD и деплой пайплайна

Урок 1 - Сборка сервисов в Docker

Шаг 01:

Вопрос 1

Какой файл является стандартным для описания сборки Docker-образа?

Подсказка: Файл должен содержать инструкции для сборки образа.

Ответ:

- **Dockerfile**
- docker-compose.yml
- Makefile

Шаг 02:

Вопрос 2

Что делает команда `docker build -t myapp .` ?

Подсказка: Подумайте о процессе создания образа и присвоении имени.

Ответ:

- **Собирает Docker-образ из Dockerfile и присваивает тег myapp**
- Запускает контейнер с именем myapp
- Удаляет старый образ myapp

Шаг 03:

Вопрос 3 Расположите шаги сборки и деплоя сервиса через Docker в правильном порядке:

1. Написание Dockerfile
2. Сборка Docker-образа
3. Запуск контейнера на локальной машине
4. Отправка образа в Docker Registry
5. Деплой на сервере или в кластер

Подсказка:

Сначала пишем инструкции, затем собираем, проверяем локально, пушим в Registry и деплоим.

Ответ:

- Написание Dockerfile
- Сборка Docker-образа
- Запуск контейнера на локальной машине
- Отправка образа в Docker Registry
- Деплой на сервере или в кластер

Шаг 04:

Вопрос 4

Сопоставьте команды Docker с их назначением:

Ответ:

- `docker run`

- `docker ps`
- `docker stop`
- `docker rm`

1. Запуск контейнера
2. Просмотр запущенных контейнеров
3. Остановка работающего контейнера
4. Удаление контейнера

🔗 Подсказка:

Помните разницу между запуском, остановкой и удалением контейнера.

- **`docker run`** — Запуск контейнера
- **`docker ps`** — Просмотр запущенных контейнеров
- **`docker stop`** — Остановка работающего контейнера
- **`docker rm`** — Удаление контейнера

🔗 Шаг 05:

Вопрос 5

Что обеспечивает непрерывную интеграцию (CI) в пайплайне с Docker?

Подсказка: Подумайте о автоматической сборке и тестировании после каждого коммита.

Ответ:

- **Автоматическая сборка и тестирование образов после коммита**
- Ручной деплой на сервер
- Удаление старых контейнеров без сборки новых

🔗 Шаг 06:

Вопрос 6 Расположите шаги CI/CD для Docker-сервиса:

1. Push кода в репозиторий
2. Сборка образа Docker
3. Запуск тестов
4. Публикация образа в Registry
5. Деплой на продакшн

🔗 Подсказка:

Сначала код в репозиторий, затем сборка, тесты, публикация и деплой.

Ответ:

- Push кода в репозиторий
- Сборка образа Docker
- Запуск тестов
- Публикация образа в Registry
- Деплой на продакшн

🔗 Шаг 07:

Вопрос 7

Сопоставьте файлы и их назначение в CI/CD пайплайне с Docker:

Ответ:

- Dockerfile
 - docker-compose.yml
 - .gitlab-ci.yml
 - .env
1. Описывает, как собрать образ
 2. Настройка сервисов и их зависимостей для локального запуска
 3. Конфигурация CI/CD пайплайна
 4. Переменные окружения для контейнеров

 Подсказка:

Каждый файл отвечает за конкретный аспект сборки, деплоя и конфигурации.

- **Dockerfile** — Описывает, как собрать образ
- **docker-compose.yml** — Настройка сервисов и их зависимостей для локального запуска
- **.gitlab-ci.yml** — Конфигурация CI/CD пайплайна
- **.env** — Переменные окружения для контейнеров

 Шаг 08:

Вопрос 8

Что помогает ускорить сборку Docker-образов при CI/CD?

Подсказка: Подумайте о повторном использовании промежуточных слоев.

Ответ:

- **Использование кэширования слоев образа**
- **Минимизация числа слоев в Dockerfile**
- Удаление кэша между сборками

 Шаг 09:

Вопрос 9 Расположите шаги деплоя микросервиса в Kubernetes через Docker:

1. Сборка Docker-образа
2. Публикация в Registry
3. Создание Deployment и Service манифестов
4. Применение манифестов через kubectl
5. Мониторинг и проверка статуса подов

 Подсказка:

Сначала создаем и публикуем образ, затем описываем деплой, применяем и проверяем.

Ответ:

- Сборка Docker-образа
- Публикация в Registry
- Создание Deployment и Service манифестов
- Применение манифестов через kubectl
- Мониторинг и проверка статуса подов

 Шаг 10:

Вопрос 10

Сопоставьте понятия CI/CD и их роль в пайплайне с Docker:

Ответ:

- CI (Continuous Integration)
 - CD (Continuous Delivery)
 - CD (Continuous Deployment)
 - Rollback
1. Автоматическая сборка и тестирование после коммита
 2. Автоматический деплой на staging для проверки
 3. Автоматический деплой на продакшн после успешного теста
 4. Возврат к предыдущей стабильной версии при проблемах

 **Подсказка:**

Разделяйте интеграцию, доставку и деплой; rollback нужен для безопасности.

- **CI (Continuous Integration)** — Автоматическая сборка и тестирование после коммита
- **CD (Continuous Delivery)** — Автоматический деплой на staging для проверки
- **CD (Continuous Deployment)** — Автоматический деплой на продакшн после успешного теста
- **Rollback** — Возврат к предыдущей стабильной версии при проблемах

Урок 2 - Настройка docker-compose для локального запуска

 **Шаг 01:**

Вопрос 1

Какой файл используется для описания многоконтейнерного приложения Docker?

Подсказка: Он позволяет запускать несколько сервисов одновременно.

Ответ:

- **docker-compose.yml**
- Dockerfile
- Makefile

 **Шаг 02:**

Вопрос 2

Что делает команда docker-compose up ?

Подсказка: Подумайте о запуске сервисов, описанных в docker-compose.yml.

Ответ:

- **Запускает все сервисы, описанные в docker-compose.yml**
- Собирает образ без запуска контейнеров
- Удаляет старые контейнеры

 **Шаг 03:**

Вопрос 3 Расположите шаги настройки локального окружения через docker-compose:

1. Создание docker-compose.yml с сервисами
2. Сборка образов сервисов
3. Запуск контейнеров с зависимостями
4. Проверка работы всех сервисов

5. Остановка контейнеров после работы

Подсказка:

Сначала описываем сервисы, потом собираем, запускаем, проверяем и в конце останавливаем.

Ответ:

- Создание docker-compose.yml с сервисами
- Сборка образов сервисов
- Запуск контейнеров с зависимостями
- Проверка работы всех сервисов
- Остановка контейнеров после работы

Шаг 04:

Вопрос 4

Сопоставьте команды docker-compose с их действием:

Ответ:

- docker-compose build
 - docker-compose up
 - docker-compose down
 - docker-compose logs
1. Сборка образов сервисов
 2. Запуск контейнеров
 3. Остановка и удаление контейнеров
 4. Просмотр логов контейнеров

Подсказка:

Команды имеют логичное соответствие своим действиям — сборка, запуск, остановка, лог.

- **docker-compose build** — Сборка образов сервисов
- **docker-compose up** — Запуск контейнеров
- **docker-compose down** — Остановка и удаление контейнеров
- **docker-compose logs** — Просмотр логов контейнеров

Шаг 05:

Вопрос 5

Для чего используют переменные окружения в docker-compose.yml?

Подсказка: Подумайте о том, как конфигурировать сервисы без изменения Dockerfile.

Ответ:

- **Для передачи конфигурации в контейнеры без изменения образа**
- Для определения версий Docker
- Для автоматического деплоя на продакшн

Шаг 06:

Вопрос 6 Расположите шаги CI/CD для docker-compose локального проекта:

1. Push кода в репозиторий
2. Сборка образов сервисов
3. Запуск контейнеров и тесты

4. Публикация образов в Registry
5. Деплой на staging/production

🔗 **Подсказка:**

CI/CD начинается с коммита, затем сборка, тесты, публикация и деплой.

Ответ:

- Push кода в репозиторий
- Сборка образов сервисов
- Запуск контейнеров и тесты
- Публикация образов в Registry
- Деплой на staging/production

🔗 **Шаг 07:**

Вопрос 7

Сопоставьте ключи docker-compose.yml с их назначением:

Ответ:

- services
 - volumes
 - networks
 - environment
1. Определение контейнеров приложения
 2. Общие тома для сохранения данных
 3. Настройка сетей для связи контейнеров
 4. Переменные окружения для контейнеров

🔗 **Подсказка:**

Каждый раздел имеет логичное назначение: сервисы, данные, сеть, конфигурация.

- **services** — Определение контейнеров приложения
- **volumes** — Общие тома для сохранения данных
- **networks** — Настройка сетей для связи контейнеров
- **environment** — Переменные окружения для контейнеров

🔗 **Шаг 08:**

Вопрос 8

Что позволяет использовать depends_on в docker-compose.yml?

Подсказка: Подумайте о порядке запуска сервисов.

Ответ:

- **Задать порядок запуска контейнеров в зависимости друг от друга**
- Автоматически масштабировать сервисы
- Удалять контейнеры после остановки

🔗 **Шаг 09:**

Вопрос 9 Расположите шаги деплоя микросервиса с docker-compose на сервер:

1. Сборка образов на локальной машине
2. Публикация образов в Registry

3. Клонирование репозитория на сервер
4. Запуск docker-compose up на сервере
5. Проверка логов и работы сервисов

 Подсказка:

Сначала собираем, публикуем, доставляем код на сервер, запускаем и проверяем работу.

Ответ:

- Сборка образов на локальной машине
- Публикация образов в Registry
- Клонирование репозитория на сервер
- Запуск docker-compose up на сервере
- Проверка логов и работы сервисов

 Шаг 10:

Вопрос 10

Сопоставьте понятия CI/CD с их ролью при работе с docker-compose:

Ответ:

- CI (Continuous Integration)
 - CD (Continuous Delivery)
 - CD (Continuous Deployment)
 - Rollback
1. Автоматическая сборка и тестирование контейнеров после коммита
 2. Автоматическая подготовка образов и деплой на staging
 3. Автоматический деплой на продакшн после успешного теста
 4. Возврат к предыдущей стабильной версии при ошибках

 Подсказка:

CI/CD помогает быстро проверять изменения и безопасно доставлять их в продакшн.

- **CI (Continuous Integration)** — Автоматическая сборка и тестирование контейнеров после коммита
- **CD (Continuous Delivery)** — Автоматическая подготовка образов и деплой на staging
- **CD (Continuous Deployment)** — Автоматический деплой на продакшн после успешного теста
- **Rollback** — Возврат к предыдущей стабильной версии при ошибках

 Урок 3 - Основы Kubernetes для микросервисов

 Шаг 01:

Вопрос 1

Что такое gRPC в контексте микросервисной архитектуры?

Подсказка: Подумайте о быстрых и типизированных коммуникациях между сервисами.

Ответ:

- **Фреймворк для удаленного вызова процедур между сервисами**
- Система управления очередями сообщений
- База данных для микросервисов

🔮 Шаг 02:

Вопрос 2

Какая роль Kafka в event-driven пайплайне?

Подсказка: Подумайте о передаче событий между сервисами без прямой связи.

Ответ:

- **Буферизация и передача событий между сервисами**
- Хранение состояния сервисов
- Автоматический деплой микросервисов

🔮 Шаг 03:

Вопрос 3 Расположите шаги передачи события через Kafka:

1. Сервис публикует событие в топик
2. Kafka сохраняет событие в брокере
3. Другой сервис подписан на топик
4. Сервис получает событие и обрабатывает его

🔮 **Подсказка:**

Сначала публикация, потом хранение, подписка и обработка.

Ответ:

- Сервис публикует событие в топик
- Kafka сохраняет событие в брокере
- Другой сервис подписан на топик
- Сервис получает событие и обрабатывает его

🔮 Шаг 04:

Вопрос 4

Сопоставьте компоненты event-driven пайплайна с их функцией:

Ответ:

- gRPC
 - Kafka
 - Redis
 - Producer
1. Удаленный вызов методов между сервисами
 2. Очередь сообщений для асинхронного обмена
 3. Кэширование и быстрый доступ к данным
 4. Отправка событий в брокер сообщений

🔮 **Подсказка:**

Подумайте, кто что выполняет в пайплайне.

- **gRPC** — Удаленный вызов методов между сервисами
- **Kafka** — Очередь сообщений для асинхронного обмена
- **Redis** — Кэширование и быстрый доступ к данным
- **Producer** — Отправка событий в брокер сообщений

🔮 Шаг 05:

Вопрос 5

Для чего нужен Consumer в event-driven архитектуре?

Подсказка: Подумайте, кто обрабатывает события после публикации.

Ответ:

- **Получает и обрабатывает события из топика**
- Публикует события в топик
- Хранит события в базе данных

🔮 Шаг 06:

Вопрос 6 Расположите шаги CI/CD пайплайна для микросервисов с gRPC и Kafka:

1. Push кода в репозиторий
2. Сборка Docker-образов сервисов
3. Запуск тестов (unit и интеграционных)
4. Публикация образов в Registry
5. Деплой на staging/production

🔮 **Подсказка:**

Сначала код, сборка, тесты, публикация и только потом деплой.

Ответ:

- Push кода в репозиторий
- Сборка Docker-образов сервисов
- Запуск тестов (unit и интеграционных)
- Публикация образов в Registry
- Деплой на staging/production

🔮 Шаг 07:

Вопрос 7

Сопоставьте команды CI/CD с их назначением для микросервисов:

Ответ:

- docker build
 - docker push
 - kubectl apply
 - helm upgrade
1. Сборка образа контейнера
 2. Публикация образа в Registry
 3. Применение манифестов Kubernetes
 4. Обновление релиза через Helm

🔮 **Подсказка:**

Каждая команда имеет специфическую функцию в CI/CD.

- **docker build** — Сборка образа контейнера
- **docker push** — Публикация образа в Registry
- **kubectl apply** — Применение манифестов Kubernetes
- **helm upgrade** — Обновление релиза через Helm

🔗 Шаг 08:

Вопрос 8

Что делает Redis в event-driven микросервисах?

Подсказка: Подумайте о хранении состояния и кэше для быстрого доступа.

Ответ:

- **Кэширует данные и хранит состояние для быстрого доступа**
- Передает события между сервисами
- Организует деплой микросервисов

🔗 Шаг 09:

Вопрос 9 Расположите шаги обработки события в пайплайне с gRPC, Kafka и Redis:

1. Сервис А отправляет событие через gRPC
2. Событие попадает в Kafka топик
3. Сервис В подписан на топик и получает событие
4. Сервис В обрабатывает событие и сохраняет результат в Redis
5. Сервис В отправляет подтверждение завершения операции

🔗 **Подсказка:**

Подумайте о последовательности передачи, обработки и сохранения события.

Ответ:

- Сервис А отправляет событие через gRPC
- Событие попадает в Kafka топик
- Сервис В подписан на топик и получает событие
- Сервис В обрабатывает событие и сохраняет результат в Redis
- Сервис В отправляет подтверждение завершения операции

🔗 Шаг 10:

Вопрос 10

Сопоставьте понятия CI/CD с их функцией в event-driven пайплайне:

Ответ:

- Continuous Integration
 - Continuous Delivery
 - Continuous Deployment
 - Rollback
1. Автоматическая сборка и тестирование сервисов после коммита
 2. Подготовка образов и деплой на staging
 3. Автоматический деплой на продакшн после успешного теста
 4. Возврат к предыдущей стабильной версии при ошибках

🔗 **Подсказка:**

Подумайте о каждом этапе: проверка, подготовка, деплой, откат.

- **Continuous Integration** — Автоматическая сборка и тестирование сервисов после коммита
- **Continuous Delivery** — Подготовка образов и деплой на staging
- **Continuous Deployment** — Автоматический деплой на продакшн после успешного теста

- **Rollback** — Возврат к предыдущей стабильной версии при ошибках

Урок 4 - CI/CD (GitHub Actions GitLab CI)

Шаг 01:

Вопрос 1

Что делает CI/CD в современном пайплайне микросервисов?

Подсказка: Подумайте о том, как автоматизировать сборку и деплой.

Ответ:

- **Автоматизирует сборку, тестирование и деплой сервисов**
- Хранит данные сервисов
- Передает события между сервисами

Шаг 02:

Вопрос 2

Для чего нужен GitHub Actions в CI/CD пайплайне?

Подсказка: Подумайте о запуске задач автоматически при коммите.

Ответ:

- **Для автоматического запуска сборки, тестов и деплоя при событиях в репозитории**
- Для хранения контейнеров
- Для передачи событий между сервисами

Шаг 03:

Вопрос 3 Расположите этапы CI/CD пайплайна в правильном порядке:

1. Push кода в репозиторий
2. Запуск тестов (unit и интеграционных)
3. Сборка Docker-образов
4. Публикация образов в Registry
5. Деплой на staging/production

 **Подсказка:**

Сначала код, затем тесты, сборка, публикация и деплой.

Ответ:

- Push кода в репозиторий
- Запуск тестов (unit и интеграционных)
- Сборка Docker-образов
- Публикация образов в Registry
- Деплой на staging/production

Шаг 04:

Вопрос 4

Сопоставьте понятия CI/CD с их назначением:

Ответ:

- Continuous Integration
- Continuous Delivery

- Continuous Deployment
- Rollback

1. Автоматическая сборка и тестирование кода после коммита
2. Подготовка к деплою на staging
3. Автоматический деплой на продакшн после успешного теста
4. Возврат к предыдущей стабильной версии при ошибках

🔗 Подсказка:

Подумайте о каждом этапе пайплайна: проверка, подготовка, деплой, откат.

- **Continuous Integration** — Автоматическая сборка и тестирование кода после коммита
- **Continuous Delivery** — Подготовка к деплою на staging
- **Continuous Deployment** — Автоматический деплой на продакшн после успешного теста
- **Rollback** — Возврат к предыдущей стабильной версии при ошибках

🔗 Шаг 05:

Вопрос 5

Что такое GitLab CI в контексте CI/CD?

Подсказка: Подумайте о встроенной системе для автоматизации сборки и деплоя.

Ответ:

- **Система автоматизации пайплайнов внутри GitLab**
- Сервис для хранения Docker-образов
- Очередь сообщений для микросервисов

🔗 Шаг 06:

Вопрос 6 Расположите шаги деплоя через GitHub Actions:

1. Push кода
2. Сборка Docker-образа
3. Запуск тестов
4. Публикация образа
5. Деплой на продакшн

🔗 Подсказка:

Сначала коммит, затем сборка, тесты, публикация и деплой.

Ответ:

- Push кода
- Сборка Docker-образа
- Запуск тестов
- Публикация образа
- Деплой на продакшн

🔗 Шаг 07:

Вопрос 7

Сопоставьте файлы конфигурации с их назначением в GitHub Actions:

Ответ:

- .github/workflows/main.yml

- .gitlab-ci.yml
- Dockerfile
- helm-chart

1. Определяет действия CI/CD в GitHub
2. Определяет пайплайн CI/CD в GitLab
3. Инструкция для сборки контейнера
4. Определяет деплой и управление приложением в Kubernetes

🔔 Подсказка:

Файлы определяют автоматизацию на разных уровнях и в разных системах.

- **.github/workflows/main.yml** — Определяет действия CI/CD в GitHub
- **.gitlab-ci.yml** — Определяет пайплайн CI/CD в GitLab
- **Dockerfile** — Инструкция для сборки контейнера
- **helm-chart** — Определяет деплой и управление приложением в Kubernetes

🔔 Шаг 08:

Вопрос 8

Какой этап CI/CD обеспечивает максимальную надежность деплоя?

Подсказка: Подумайте о проверке перед продакшн деплоем.

Ответ:

- **Запуск автоматических тестов и проверка качества кода**
- Сборка Docker-образа
- Публикация образа в Registry

🔔 Шаг 09:

Вопрос 9 Расположите шаги автоматического отката при ошибке деплоя:

1. Мониторинг продакшн
2. Обнаружение ошибки
3. Запуск rollback
4. Подтверждение успешного отката

🔔 Подсказка:

Сначала мониторим, потом фиксируем, затем откатываем и подтверждаем.

Ответ:

- Мониторинг продакшн
- Обнаружение ошибки
- Запуск rollback
- Подтверждение успешного отката

🔔 Шаг 10:

Вопрос 10

Сопоставьте термины CI/CD с их примерами:

Ответ:

- Pipeline
- Job

- Stage
- Artifact

1. Полный процесс от коммита до деплоя
2. Отдельная задача, например тест или сборка
3. Этап пайплайна, объединяющий несколько задач
4. Результат выполнения задачи или сборки

🔔 Подсказка:

Подумайте о структуре CI/CD: процессы, этапы, задачи и результаты.

- **Pipeline** — Полный процесс от коммита до деплоя
- **Job** — Отдельная задача, например тест или сборка
- **Stage** — Этап пайплайна, объединяющий несколько задач
- **Artifact** — Результат выполнения задачи или сборки

📖 Урок 5 - Практика развернуть пайплайн в Docker Compose.

🔔 Шаг 01:

Вопрос 1

Что необходимо для запуска микросервисов в Docker Compose?

Подсказка: Подумайте о базовом файле конфигурации.

Ответ:

- **docker-compose.yml**
- Dockerfile
- .gitlab-ci.yml

🔔 Шаг 02:

Вопрос 2

Какая команда запускает все сервисы, описанные в docker-compose.yml?

Подсказка: Команда включает ключевое слово 'up'.

Ответ:

- **docker-compose up -d**
- docker run
- docker build

🔔 Шаг 03:

Вопрос 3 Расположите шаги запуска пайплайна в Docker Compose:

1. Написание docker-compose.yml
2. Сборка образов сервисов
3. Запуск сервисов командой docker-compose up
4. Проверка состояния контейнеров командой docker-compose ps

🔔 Подсказка:

Сначала конфигурация, затем сборка, запуск и проверка.

Ответ:

- Написание docker-compose.yml

- Сборка образов сервисов
- Запуск сервисов командой `docker-compose up`
- Проверка состояния контейнеров командой `docker-compose ps`

🔗 Шаг 04:

Вопрос 4

Сопоставьте элементы `docker-compose.yml` с их назначением:

Ответ:

- `services`
 - `volumes`
 - `networks`
 - `depends_on`
1. Определение контейнеров для запуска
 2. Настройка общих томов для данных
 3. Создание виртуальных сетей между сервисами
 4. Задание порядка запуска сервисов

🔗 Подсказка:

Подумайте о структуре конфигурации и связях сервисов.

- **`services`** — Определение контейнеров для запуска
- **`volumes`** — Настройка общих томов для данных
- **`networks`** — Создание виртуальных сетей между сервисами
- **`depends_on`** — Задание порядка запуска сервисов

🔗 Шаг 05:

Вопрос 5

Как проверить логи всех контейнеров в Docker Compose?

Подсказка: Используйте команду с ключевым словом `'logs'`.

Ответ:

- **`docker-compose logs -f`**
- `docker ps`
- `docker exec`

🔗 Шаг 06:

Вопрос 6 Расположите шаги обновления сервиса в Docker Compose:

1. Остановка контейнера
2. Сборка нового образа
3. Запуск контейнера с новым образом
4. Проверка работы сервиса

🔗 Подсказка:

Сначала остановка, сборка, запуск и проверка.

Ответ:

- Остановка контейнера
- Сборка нового образа

- Запуск контейнера с новым образом
- Проверка работы сервиса

🔗 Шаг 07:

Вопрос 7

Сопоставьте команды Docker Compose с их назначением:

Ответ:

- `docker-compose up -d`
 - `docker-compose down`
 - `docker-compose ps`
 - `docker-compose exec`
1. Запуск сервисов в фоновом режиме
 2. Остановка и удаление контейнеров
 3. Показ состояния контейнеров
 4. Выполнение команды внутри контейнера

🔗 Подсказка:

Подумайте, что делает каждая команда в повседневной работе с Compose.

- **`docker-compose up -d`** — Запуск сервисов в фоновом режиме
- **`docker-compose down`** — Остановка и удаление контейнеров
- **`docker-compose ps`** — Показ состояния контейнеров
- **`docker-compose exec`** — Выполнение команды внутри контейнера

🔗 Шаг 08:

Вопрос 8

Что делает директива 'build' в `docker-compose.yml`?

Подсказка: Подумайте о создании контейнера из Dockerfile.

Ответ:

- **Указывает путь к Dockerfile для сборки образа**
- Задаёт сеть между контейнерами
- Определяет переменные окружения

🔗 Шаг 09:

Вопрос 9 Расположите шаги масштабирования сервиса в Docker Compose:

1. Редактирование `docker-compose.yml` с указанием количества реплик
2. Применение изменений командой `docker-compose up -d`
3. Проверка состояния сервисов

🔗 Подсказка:

Сначала редактируем, затем применяем и проверяем.

Ответ:

- Редактирование `docker-compose.yml` с указанием количества реплик
- Применение изменений командой `docker-compose up -d`
- Проверка состояния сервисов

Шаг 10:

Вопрос 10

Сопоставьте типичные проблемы при работе с Docker Compose с их решениями:

Ответ:

- Проблема с портами
 - Контейнер не запускается
 - Ошибка связи между сервисами
 - Образы устарели
1. Изменить маппинг портов в docker-compose.yml
 2. Проверить логи и Dockerfile
 3. Использовать networks и depends_on
 4. Собрать новые образы с актуальным кодом

Подсказка:

Каждая проблема имеет стандартное решение через конфигурацию или пересборку образов.

- **Проблема с портами** — Изменить маппинг портов в docker-compose.yml
- **Контейнер не запускается** — Проверить логи и Dockerfile
- **Ошибка связи между сервисами** — Использовать networks и depends_on
- **Образы устарели** — Собрать новые образы с актуальным кодом

Модуль 10 - Итоговый проект

Урок 1 - Реализация полного пайплайна Client → gRPC API → Kafka → Worker

Шаг 01:

Вопрос 1

Какой компонент отвечает за приём запросов от клиента в нашем пайплайне?

Подсказка: Это точка входа для всех gRPC вызовов.

Ответ:

- **gRPC API**
- Worker
- Kafka Topic

Шаг 02:

Вопрос 2

Что делает Kafka в нашем пайплайне?

Подсказка: Подумайте о передаче сообщений между сервисами.

Ответ:

- **Передаёт сообщения от gRPC API к Worker**
- Обработывает HTTP запросы клиентов
- Сохраняет данные в БД напрямую

🔗 Шаг 03:

Вопрос 3 Расположите шаги обработки сообщения в правильном порядке:

1. Клиент отправляет запрос
2. gRPC API получает и валидирует запрос
3. Сообщение публикуется в Kafka Topic
4. Worker получает сообщение и обрабатывает его

🔗 Подсказка:

Подумайте о последовательности движения данных от клиента до Worker.

Ответ:

- Клиент отправляет запрос
- gRPC API получает и валидирует запрос
- Сообщение публикуется в Kafka Topic
- Worker получает сообщение и обрабатывает его

🔗 Шаг 04:

Вопрос 4

Сопоставьте компоненты пайплайна с их функциями:

Ответ:

- Client
 - gRPC API
 - Kafka
 - Worker
1. Отправляет запросы в пайплайн
 2. Принимает запросы и валидирует их
 3. Передаёт сообщения между сервисами
 4. Обрабатывает сообщения и выполняет задачу

🔗 Подсказка:

Подумайте о роли каждого элемента в потоке данных.

- **Client** — Отправляет запросы в пайплайн
- **gRPC API** — Принимает запросы и валидирует их
- **Kafka** — Передаёт сообщения между сервисами
- **Worker** — Обрабатывает сообщения и выполняет задачу

🔗 Шаг 05:

Вопрос 5

Какой формат сообщений рекомендуется использовать при отправке данных через Kafka в нашем пайплайне?

Подсказка: Подумайте о простом и сериализуемом формате.

Ответ:

- **JSON или Protobuf**
- HTML
- XML без схемы

🔗 Шаг 06:

Вопрос 6 Расположите шаги от публикации до обработки сообщения:

1. Сообщение отправлено в Kafka Topic
2. Worker подписан на Topic и получает сообщение
3. Worker выполняет бизнес-логику
4. Результат обработки отправляется в БД или другой сервис

🔗 Подсказка:

Сначала публикация, затем получение, обработка и отправка результата.

Ответ:

- Сообщение отправлено в Kafka Topic
- Worker подписан на Topic и получает сообщение
- Worker выполняет бизнес-логику
- Результат обработки отправляется в БД или другой сервис

🔗 Шаг 07:

Вопрос 7

Сопоставьте типичные ошибки пайплайна с их решениями:

Ответ:

- Сообщение потеряно в Kafka
 - Worker не обрабатывает сообщения
 - gRPC API возвращает ошибку
 - Сбой в логике обработки
1. Включить репликацию и хранение сообщений
 2. Проверить подписку и подключение к Topic
 3. Проверить валидаторы и сериализацию данных
 4. Использовать unit-тесты и логирование

🔗 Подсказка:

Каждая ошибка имеет стандартное решение через конфигурацию или код.

- **Сообщение потеряно в Kafka** — Включить репликацию и хранение сообщений
- **Worker не обрабатывает сообщения** — Проверить подписку и подключение к Topic
- **gRPC API возвращает ошибку** — Проверить валидаторы и сериализацию данных
- **Сбой в логике обработки** — Использовать unit-тесты и логирование

🔗 Шаг 08:

Вопрос 8

Что необходимо сделать, чтобы добавить новый тип события в пайплайн?

Подсказка: Подумайте о всех компонентах: gRPC, Kafka, Worker.

Ответ:

- **Обновить gRPC API, создать новый Topic в Kafka и Worker обработчик**
- Только добавить обработку в Worker
- Только изменить Kafka Topic

Шаг 09:

Вопрос 9 Расположите шаги деплоя пайплайна:

1. Сборка Docker образов для всех сервисов
2. Запуск сервисов с docker-compose или Kubernetes
3. Проверка логов и состояния сервисов
4. Запуск интеграционных тестов

Подсказка:

Сначала сборка, затем запуск, проверка и тестирование.

Ответ:

- Сборка Docker образов для всех сервисов
- Запуск сервисов с docker-compose или Kubernetes
- Проверка логов и состояния сервисов
- Запуск интеграционных тестов

Шаг 10:

Вопрос 10

Сопоставьте проблемы интеграционного пайплайна с решениями:

Ответ:

- Ошибка сериализации данных
 - Worker не получает сообщения
 - gRPC API не принимает новые запросы
 - Сообщения дублируются
1. Использовать правильный формат (Protobuf/JSON)
 2. Проверить подписку и соединение с Kafka
 3. Обновить API и протестировать новые методы
 4. Включить уникальный ключ или идемпотентность

Подсказка:

Каждая проблема требует внимания к компоненту и его конфигурации.

- **Ошибка сериализации данных** — Использовать правильный формат (Protobuf/JSON)
- **Worker не получает сообщения** — Проверить подписку и соединение с Kafka
- **gRPC API не принимает новые запросы** — Обновить API и протестировать новые методы
- **Сообщения дублируются** — Включить уникальный ключ или идемпотентность

Урок 2 - Тестирование под нагрузкой (JMeter/locust)

Шаг 01:

Вопрос 1

Какой инструмент используется для нагрузочного тестирования HTTP сервисов?

Подсказка: Популярный инструмент с графическим интерфейсом и возможностью записи сценариев.

Ответ:

- **JMeter**
- Redis

- Kafka

Шаг 02:

Вопрос 2

Для чего используют Locust в тестировании?

Подсказка: Подумайте о Python и масштабируемости сценариев.

Ответ:

- **Создание сценариев нагрузки с помощью Python**
- Мониторинг логов Kafka
- Управление Docker контейнерами

Шаг 03:

Вопрос 3 Расположите шаги нагрузочного тестирования в правильном порядке:

1. Создание сценария нагрузки
2. Запуск теста
3. Сбор метрик и логов
4. Анализ результатов

 **Подсказка:**

Сначала пишем сценарий, потом запускаем тест, собираем данные и анализируем.

Ответ:

- Создание сценария нагрузки
- Запуск теста
- Сбор метрик и логов
- Анализ результатов

Шаг 04:

Вопрос 4

Сопоставьте термины нагрузочного тестирования с их определениями:

Ответ:

- TPS
 - Concurrent Users
 - Response Time
 - Error Rate
1. Количество транзакций в секунду
 2. Количество одновременных пользователей
 3. Время ответа сервиса на запрос
 4. Процент неудачных запросов

 **Подсказка:**

Подумайте о метриках, которые отслеживают производительность сервиса.

- **TPS** — Количество транзакций в секунду
- **Concurrent Users** — Количество одновременных пользователей
- **Response Time** — Время ответа сервиса на запрос
- **Error Rate** — Процент неудачных запросов

Шаг 05:

Вопрос 5

Что является основной целью нагрузочного тестирования?

Подсказка: Подумайте о поведении системы при высокой нагрузке.

Ответ:

- **Проверка стабильности и производительности системы**
- Изучение функциональных требований
- Создание графического интерфейса

Шаг 06:

Вопрос 6 Расположите шаги подготовки JMeter теста:

1. Создание Thread Group
2. Добавление HTTP Request
3. Настройка Listener для сбора данных
4. Запуск теста

 **Подсказка:**

Сначала создаём группу потоков, затем добавляем запрос, listener и запускаем тест.

Ответ:

- Создание Thread Group
- Добавление HTTP Request
- Настройка Listener для сбора данных
- Запуск теста

Шаг 07:

Вопрос 7

Сопоставьте типовые проблемы нагрузочного тестирования с решениями:

Ответ:

- Сбой сервиса при высокой нагрузке
 - Низкая точность метрик
 - Тест не стартует
 - Ошибки в сценарии
1. Увеличить ресурсы или оптимизировать сервис
 2. Настроить правильный Listener или мониторинг
 3. Проверить настройки Thread Group и подключения
 4. Исправить код сценария

 **Подсказка:**

Каждая проблема имеет простое решение через конфигурацию или исправление сценария.

- **Сбой сервиса при высокой нагрузке** — Увеличить ресурсы или оптимизировать сервис
- **Низкая точность метрик** — Настроить правильный Listener или мониторинг
- **Тест не стартует** — Проверить настройки Thread Group и подключения
- **Ошибки в сценарии** — Исправить код сценария

Шаг 08:

Вопрос 8

Какой параметр JMeter позволяет увеличить число одновременных пользователей?

Подсказка: Это параметр внутри Thread Group.

Ответ:

- **Number of Threads (users)**
- Ramp-Up Period
- Loop Count

Шаг 09:

Вопрос 9 Расположите этапы анализа результатов нагрузки:

1. Сбор графиков и логов
2. Сравнение с SLA
3. Определение узких мест
4. Подготовка отчёта

 **Подсказка:**

Сначала собираем данные, потом анализируем и готовим выводы.

Ответ:

- Сбор графиков и логов
- Сравнение с SLA
- Определение узких мест
- Подготовка отчёта

Шаг 10:

Вопрос 10

Сопоставьте проблемы при нагрузочном тестировании с возможными улучшениями:

Ответ:

- Высокое время отклика
 - Ошибки 500
 - Тесты нестабильны
 - Снижение пропускной способности
1. Оптимизация кода и увеличение ресурсов
 2. Исправление багов сервиса
 3. Перепроверка сценариев и настройки среды
 4. Балансировка нагрузки и масштабирование

 **Подсказка:**

Каждую проблему можно решить корректировкой инфраструктуры или кода.

- **Высокое время отклика** — Оптимизация кода и увеличение ресурсов
- **Ошибки 500** — Исправление багов сервиса
- **Тесты нестабильны** — Перепроверка сценариев и настройки среды
- **Снижение пропускной способности** — Балансировка нагрузки и масштабирование